

Архитектура компьютера и основы ОС Linux

Ядро операционной системы

Что такое операционная система. Загрузка ПК на примере MS DOS. Что такое ядро. Архитектура ядер (монолитное, гибридное, микроядро). Примеры ОС с разными ядрами.

Оглавление

[Что такое операционная система](#)

[Архитектура ОС на примере MS DOS](#)

[Драйверы MS DOS](#)

[Шрифты в MS DOS](#)

[Кодировки](#)

[Управляющие и псевдографические символы](#)

[Модульность DOS](#)

[Однозадачность](#)

[Оболочки](#)

[Утилиты](#)

[Архитектура ядра операционной системы](#)

[Процесс](#)

[Кольца защиты процессора](#)

[Системные вызовы](#)

[Архитектуры ядер](#)

[Монолитное ядро](#)

[Модульное ядро](#)

[Микроядро](#)

[Экзоядро](#)

[Наноядро](#)

[Гибридное ядро](#)

[Архитектура ядра Linux](#)

[Цифры и факты](#)

[Системные вызовы](#)

[Управление памятью](#)

[Управление процессами](#)

[Сетевая подсистема](#)

[VFS — виртуальная файловая система](#)

[Драйверы файловых систем](#)

[Страничный кеш](#)

[Уровень блочного ввода-вывода](#)

[Планировщик ввода-вывода](#)

[Обработка прерываний](#)

[Драйверы устройств](#)

[Практическое задание](#)

[Дополнительные материалы](#)

[Используемая литература](#)

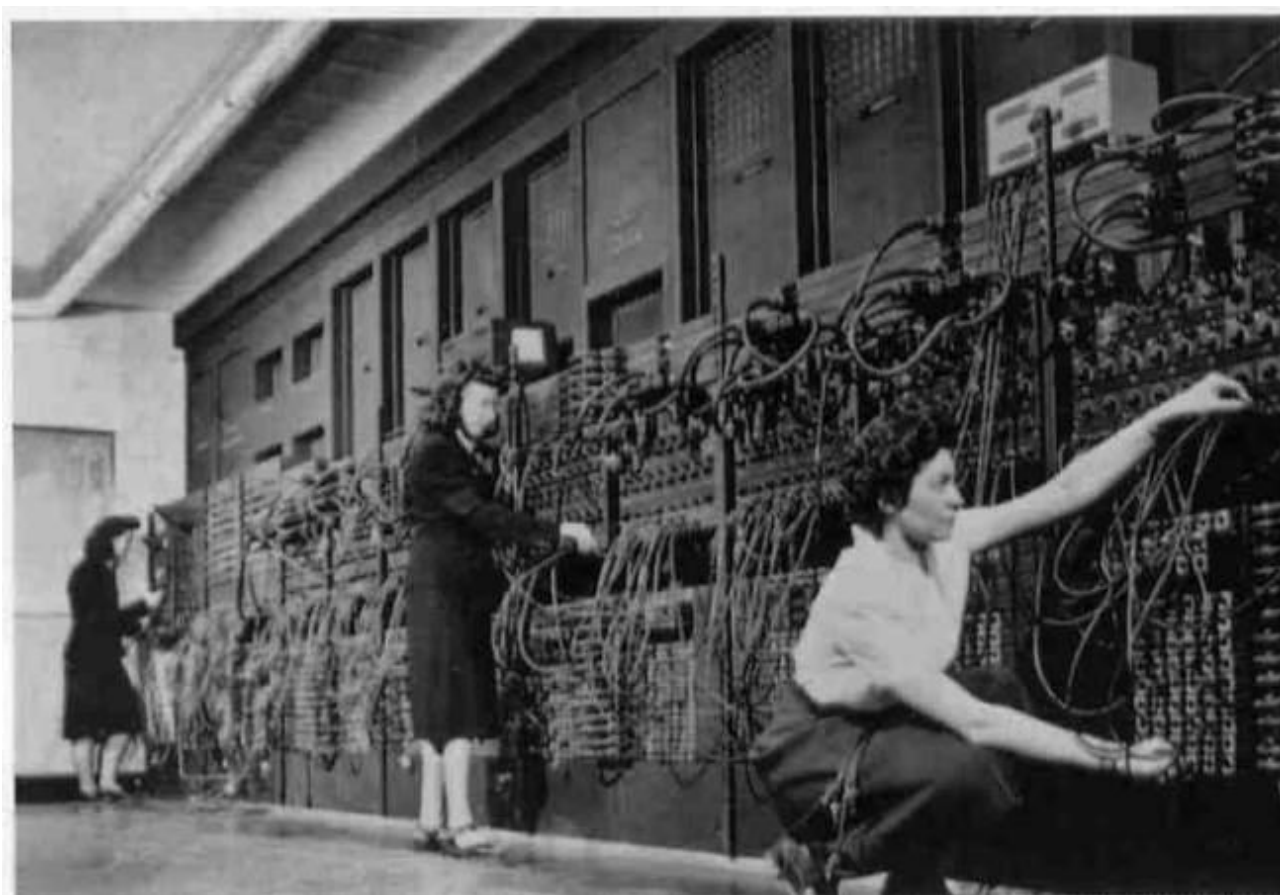
Что такое операционная система

Самые первые прототипы компьютеров (калькуляторы, машины для шифрования) не требовали операционных систем. Они уже были реализованы с заложенной в них программой работы. Фиксированные данные на входе, ожидаемый результат — на выходе.

Когда была сформулирована концепция Тьюринг-машины и появились компьютеры универсального назначения, стало ясно, что имея одну машину, меняя на ней программы, можно получить разные аппараты. И сейчас для нас банкомат, терминал оплаты или электронной очереди выглядят как разные машины, хотя на самом деле за ними стоит один и тот же универсальный компьютер (правда, с дополнительной периферией, например для работы с купюрами и штрихкодами).

Так же был изобретен процессор Intel 4004. Стало ясно, что можно выпускать разные калькуляторы, меняя всего лишь программу (прошивку), а не устройством.

Первоначально для ввода программы требовалось много усилий.



Так специалисты, переключая кабели, перепрограммируют ЭНИАК.

Это долго, неудобно и не позволяет использовать разные программы одновременно на компьютере. Кроме того, программа взаимодействует с оборудованием напрямую.

Так появилось понимание, что нужно иметь возможность запускать разные программы и управлять их работой. Первыми предшественниками операционных систем были пакетные режимы, которые запускали программы по очереди и по расписанию (.batch-файлы и скрипты и сейчас позволяют запускать программы по очереди, это наследие тех эпох).

Современная операционная система состоит из комплекса программ:

- загрузчика операционной системы;

- ядра операционной системы:
 - планировщика процессов, обработчика прерываний и системных вызовов;
 - драйверов — компонентов, отвечающих за взаимодействие с оборудованием.
- сервисов (демонов) — программ, постоянно работающих и выполняющих задачи в фоновом режиме (выполнение программ по расписанию, ведение логов, сервис печати и т. д.);
- утилит — служебных программ для работы с диском, файлами, памятью, для диагностики и управления ресурсами, в том числе и пакетных менеджеров для установки и удаления программ;
- Оболочек:
 - командных оболочек, обладающих синтаксисом, который позволяет как выполнять команды в интерактивном режиме, так и в пакетном (язык командной оболочки приближается к языкам программирования — shell, bash, powershell);
 - файловых менеджеров (Norton Commander, Midnight Commander);
 - графических оболочек.
- языков программирования и сред разработки.

На машину с операционной системой можно устанавливать прикладное программное обеспечение. Простейшие программы (а иногда даже игры) уже входят в состав поставки дистрибутива современных ОС. Очень часто успех той или иной ОС и определяется наличием соответствующих программ. Так что своим успехом Windows во многом обязана пакету Microsoft Office.

Полноценная операционная система нужна не всегда. Если это специализированное устройство, то в ПЗУ (постоянном запоминающем устройстве) может находиться прошивка, содержащая всего лишь одну программу для микроконтроллера. Самые простые коммутаторы и маршрутизаторы могут и быть такими компьютерами с одной программой. Но более сложные — это полноценные компьютеры со своей, пусть и специализированной, но операционной системой.

Наличие операционной системы не обязательно для выполнения программы. В принципе, любая программа может быть написана для работы сразу после запуска компьютера. Это неудобно и неоправданно, но есть и случаи, когда программы работают сразу после запуска, без операционной системы.

К таким можно отнести:

- программу POST в составе ПЗУ;
- код загрузочного сектора, программы **MBR** и **EBR** в начале загрузочного диска (в том числе код, выдающий сообщение «Disk boot failure, insert system disk and press any key» и его вариации, например от **ndd**);
- загрузчики операционной системы: **ntldr**, **LILO**, **GRUB**;
- программы для тестирования неисправностей (например, **Memtest 86+**);
- прошивку интерпретаторов **BASIC** на заре появления и распространения микрокомпьютеров.

В то же время строгой разницы между программой и операционной системой нет. По факту, единолично работающая на процессоре программа тоже является своего рода операционной системой. Она управляет ресурсами, взаимодействует с устройствами.

Но некоторые из таких программ могут предоставлять более привычные возможности операционных систем. Загрузчик ОС, например **GRUB**, сам по сложности приближается к простой однозначной операционной системе (похожий на **DOS** и даже превосходящей ее): имеет модули, оболочку, команды.

Основным компонентом операционной системы является ядро. Как правило, код ядра начинает выполняться при передаче управления операционной системе. Ядро запускает свои компоненты. Реализует интерфейсы системных вызовов (через механизм прерываний либо в 32-битной системе через инструкцию **sysenter**, либо в 64-битных системах через инструкцию **syscall**). При этом оно предоставляет единообразный API — интерфейс для доступа к файлам, ресурсам, устройствам, — запускает управление многозадачностью (если такое есть), механизмы управления памятью и SWAP-дискон или файлом подкачки. Одного ядра для работы недостаточно, но и без него операционная система превращается в прикладную программу, исполняющуюся в монопольном режиме. Даже такие простые по современным меркам операционные системы, как **CP/M** и **MS DOS**, обладали ядром.

Конечный пользователь видит не ядро, а оболочку. Это либо оболочка, реализующая командную строку (**command.com** в **DOS**, **sh** или **bash** в **Linux**), либо графическая оконная оболочка (встроенная в систему **Windows** оболочка с разным типом окружения рабочего стола поверх **X-Window system** в **Linux** и некоторых других **UNIX**-подобных системах — **KDE**, **XDE**, **Gnome** или **Unity**).

Архитектура ОС на примере MS DOS

Если рассмотреть содержание дискеты с **MS DOS**, в простейшем варианте там будут следующие файлы:

- **io.sys**;
- **msdos.sys**;
- **config.sys**;
- **autoexec.bat**;
- **command.com**.

Особого загрузчика в **MS DOS** не было, поэтому файлы **io.sys** и **msdos.sys** должны были располагаться в начальной области диска. Простым копированием этих файлов сделать дискету загрузочной не получилось бы — для этого использовалась утилита **sys.com**.

Предположим, что дискета вставлена в дисковод, компьютер включен. Система еще не начала читать загрузочный сектор дискеты, а уже выполнена программа **POST** из состава **BIOS** (**basic input output system**), определена таблица векторов прерываний, уже реализованы базовые возможности управления устройствами через прерывания **BIOS**. Дальше можно загрузить единственную программу, которая сможет работать даже без ОС, используя прерывания **BIOS** или взаимодействуя с оборудованием напрямую.

io.sys формально представляет собой расширение функций **BIOS**. Он перекрывает и дополняет функции, связанные с вводом и выводом. На самом деле это более сложный и важный компонент операционной системы. После замещения функций **BIOS** (по существу, **io.sys** содержит низкоуровневые драйверы операционной системы **MS DOS**) **io.sys** читает текстовый файл **config.sys**. Этот **INI**-подобный файл содержит некоторые настройки системы (например, разрешение или запрет прерывания программ по **CTRL-C**, определение количества открытых файлов) и список загружаемых драйверов (управления памятью, русификаторы и т. д.). **io.sys** загружает драйверы в память, после чего передает управление **msdos.sys**.

msdos.sys — ядро операционной системы, которое предоставляет интерфейс API для прикладных программ. **msdos.sys** определяет прерывания, которые обрабатываются **MS DOS**. В частности, это прерывания 0×20 и 0×21 .

Следующие примеры демонстрируют обращения DOS-программ к находящемуся в памяти **msdos.sys** через API.

Пример COM-программы для MS DOS:

```
.386
model tiny                /Указание модели памяти
Code segment use16        /Начало описания сегмента кода
ASSUME cs:Code, ds:Code   /Ассоциация регистров с сегментом
    org 100h              /Генерация смещения на 256 байт
start:                    /Метка начала программы
    push cs               /Запись регистра CS в стек
    pop ds                /Загрузка регистра DS значением из стека
    mov dx, offset mess   /Помещение в DX смещения строки mess
    mov ah, 09h           /Запись в AH номера функции вывода строки
    int 21h               /Вызов сервиса MS DOS
    int 20h               /Завершение COM программы в MS DOS
mess db 'Hello world!','$' /Объявление строки
Code ends                 /Завершение описания строки
end start
```

Пример EXE-программы для MS DOS:

```
.386
model small                /Указание модели памяти
Stack SEGMENT STACK use16  /Объявление сегмента стека
    ASSUME ss:Stack        /Ассоциация регистра SS с сегментом стека
    DB 100h dup(?)         /Резервирование 256 байт под стек
Stack ENDS                 /Завершение описания сегмента стека
Data SEGMENT use16         /Объявление сегмента данных
    ASSUME ds:Data         /Ассоциирование регистра DS с сегментом данных
mess db 'Hello world!','$' /Объявление строки
Data ENDS                  /Завершение описания сегмента данных
Code SEGMENT use16         /Объявление сегмента кода
    ASSUME cs:Code         /Ассоциирование регистра CS с сегментом кода
start:                     /Метка начала программы
    mov ax, seg mess       /Загрузка в AX адреса сегмента строки mess
    mov ds, ax             /Запись в DS значения AX
    mov dx, offset mess    /Запись в DX смещения строки mess
    mov ah, 09h            /Запись в AH номера функции вывода строки
    int 21h                /Вызов сервиса MS DOS
    mov ax, 4c00h          /Запись в AX функции завершения программы
    int 21h                /Завершение EXE-программы в MS DOS
Code ENDS                  /Завершение описания сегмента данных
end start
```

После того как **msdos.sys** загрузился и переопределил необходимые прерывания в таблице прерываний, он остается в памяти, но управление вновь возвращается в **io.sys**.

Далее **io.sys** загружает и передает управление файлу **command.com** (или иной обложке, если она указана в параметре **shell** файла **config.sys**).

При таком случае запуска (а не запуска копии или запуска в режиме «Выход в DOS», доступного во многих сложных программах под DOS) **command.com** выполняет файл **autoexec.bat**, а затем переходит в режим командной строки. Система загружена.

Остальные части операционной системы — это драйверы и утилиты.

Драйверы — это программы **sys** и **exe**, загружаемые, как правило (но не обязательно), из **config.sys**, которые оставались в памяти постоянно (резидентно) и предоставляли измененный интерфейс для доступа к оборудованию или иные программные удобства. Как правило, драйверы перехватывают одно из прерываний для предоставления нужных функций.

Утилиты — программы, которые обычно вызываются как команды из командной строки. У них также может быть интерфейс.

Также в MS DOS присутствовали дополнительные программы. Вместе с первыми версиями MS DOS поставлялись версии **basic** или **gwbasic** — популярный интерпретатор языка Бейсик.

Разумеется, для полноценной работы необходимы драйверы, утилиты и прикладные программы.

Драйверы MS DOS

Драйверы в MS DOS загружались в формате **sys** или **exe**, как правило, на основе его упоминания в файле **config.sys**. Формат **sys** похож на **com**, но содержит дополнительную информацию о таблице размещения драйверов. Более того, драйвер мог быть загружен в **autoexec.bat** или вообще из командной строки (перехватив прерывание и при завершении работы сообщив DOS, что не следует выгружать программу из памяти, то есть как резидентная программа). Никаких механизмов защиты не

предполагалось — по существу, операционная система и прикладные программы работали в одном пространстве, поэтому ошибка в программе приводила к зависанию всей системы. DOS — довольно простая система, поэтому программы часто использовали непосредственно работу с оборудованием или применением специальных программ-расширителей, которые сами можно воспринимать как отдельные операционные системы. Кроме того, хотя Windows 3.11 часто обозначается как оболочка, она может загружаться и обратно возвращать управление в DOS — фактически это отдельная операционная система. Драйверы в формате **sys** могут быть, как мы потом увидим и в Linux, символьными и блочными.

В MS DOS не различались пространства ядра и пользователя. Любая пользовательская программа могла оставаться в памяти как драйвер (чем пользовались компьютерные вирусы) или использовать собственные драйверы.

Часто используемые в MS DOS драйверы (названия могут меняться):

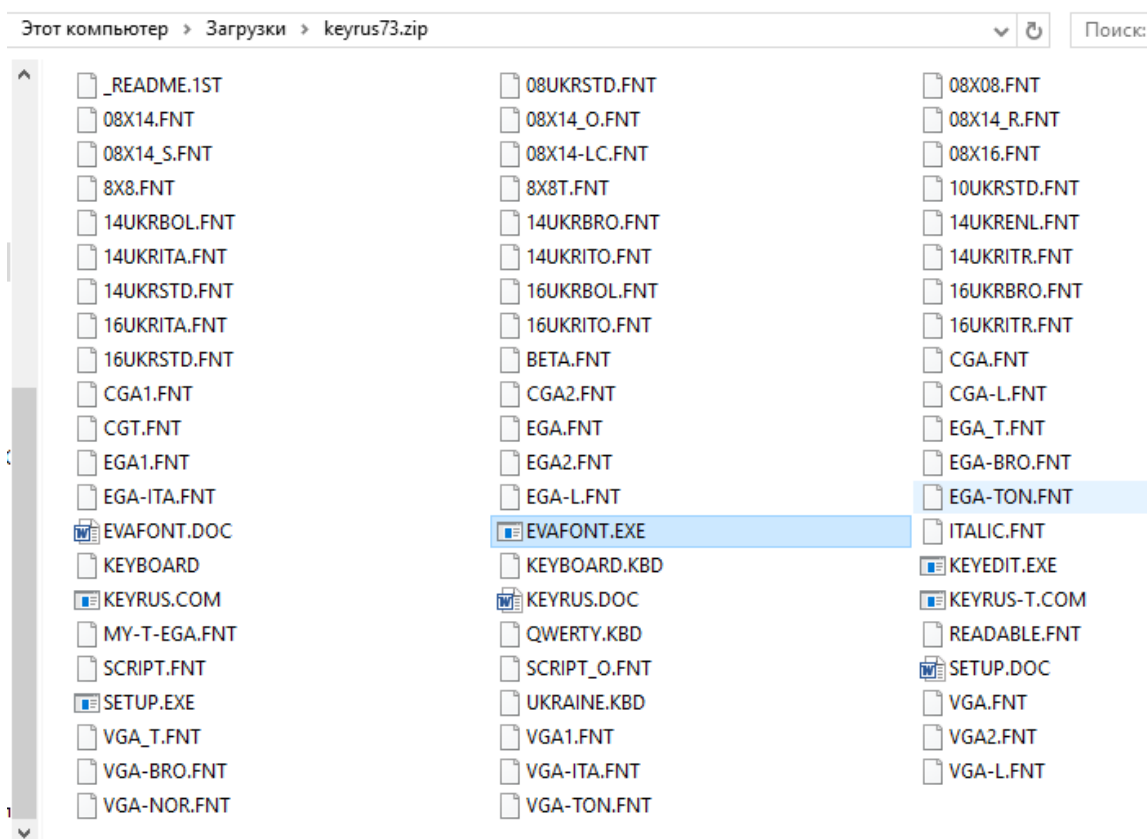
- **mouse.sys/mouse.com** — драйвер мыши. Содержал обработчик прерывания от мыши, отрисовывал курсор (в текстовом виде инверсией цветов позиции, в графическом — в виде стрелки), отвечал за функции включения и выключения курсора, получения его координат и факта срабатывания клика мыши. По умолчанию поддержки мыши в MS DOS не было. Для управления ею требовалось, чтобы прикладные программы могли работать с драйвером мыши (включать курсор, реагировать на клик);
- **cdrom.sys/cdromdrv.sys/cddriver.sys/mscdex.exe** — драйвер CD ROM. Делал доступным привод CD ROM, который получал свободную букву устройства (например, E:, а диапазон доступных букв можно было изменить в **config.sys**);
- **ramdrive.sys/ramdrive.exe/ramdisk.exe** — организовал использование части оперативной памяти как жесткого диска. При выключении системы все данные, «записанные» на такой диск, разумеется, пропадали. Драйвер применялся для возможности запуска ОС с дискет, когда сжатые на дискете файлы распаковывались в RAM-диск и запускались оттуда;
- **KeyRus.exe/kb.com** — драйвер-русификатор **KeyRus**, написанный студентом Дмитрием Гуртяком в 1989 году. Популярная на постсоветском пространстве программа заменяла точечные шрифты на русифицированные, а также перехватывала прерывание от клавиатуры для отслеживания переключения раскладки (обычно правый Ctrl переключал РУС/ЛАТ, а правый Alt позволял набирать псевдографику, что могло быть полезно для ASCII-артов).

Шрифты в MS DOS

Рассмотрим подробнее момент, связанный с русификацией. Русификатор выполнял две задачи: загружал шрифт и обрабатывал прерывания от клавиатуры. Обработка прерываний от внешнего устройства — обычная ситуация для драйверов устройств.

Каким образом хранились шрифты в MS DOS? Изначально в микрокомпьютерах применялись (и сейчас применяются для определенных задач) точечные шрифты. Их таблицы хранились в памяти и выглядели одинаково, механизм формирования нового символа для шрифта идентичен и для ZX-Spectrum (матрица 8 на 8), и для IBM PC (как правило, матрица 8 на 14, но могут быть и другие размеры).

Скачав на мемориальной странице Дмитрия Гуртяка архив с **keyrus**, увидим множество файлов **FNT**.



Это связано с тем, что раскладки могут отличаться как кодировкой и поддерживаемым языком, так и разрешением экрана.

В DOS были доступны несколько режимов экрана, поддерживаемых драйвером видеоадаптера. Текстовый режим, как правило, содержал 25 строк по 80 столбцов, поддерживалось 16 цветов. В таком режиме можно было только выводить символы. Таблица содержала 256 символов в формате матриц 8 на 14 (или 8 на 16).

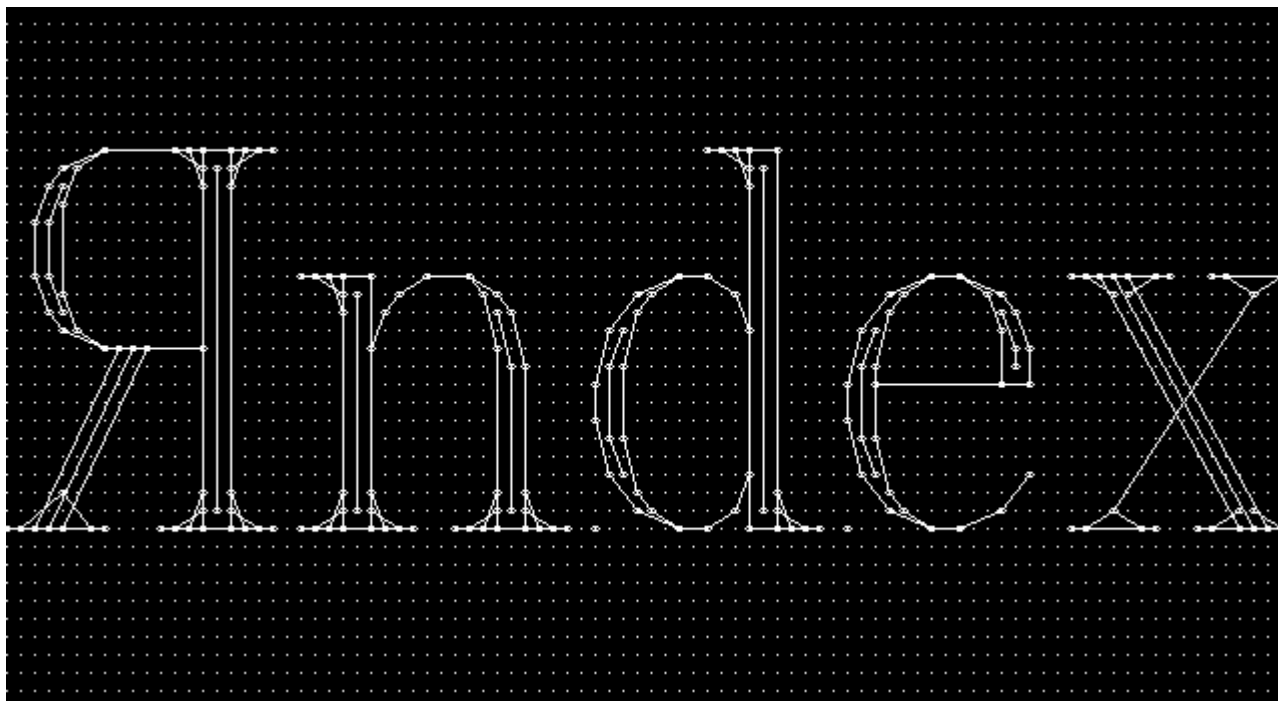
При печати строк с переводом строки в 24 и 25 строки все текстовые строки сдвигались вверх, содержимое первой и второй при этом не сохранялось.

Были и другие режимы: например, 50 строк на 80 столбцов. Графические режимы, как правило, работали в режиме пиксельного вывода (320 × 240, 640 × 480, 800 × 600), имитировали EGA- и VGA-адаптеры, обладали разной цветностью: монохромные, 4-цветные CGA, 16-цветные EGA и т. д. Режим можно было переключить запросом соответствующего прерывания. Несмотря на работу с точками, действовали те же функции печати, что и в текстовом режиме. Однако теперь в памяти не сохранялись коды символов, а сами символы выводились точно. Используя посимвольную печать или изменив положение текстового курсора, возможно было напечатать символ там, где до этого символ уже был. При печати в 24 и 25 строке текст также сдвигался, при этом смещалась и графика с потерей верхних позиций не только текста, но и рисунка. Графический режим позволял работать с собственными методами печати — можно было не обращаться к операционной системе, а самостоятельно поточно отрисовать символы: например, брать их из того же файла **fnt** или использовать нестандартные. Таким образом, технически можно было использовать даже Unicode, если подготовить соответствующий точечный шрифт.

Альтернативой было использование векторных шрифтов. Например, в Turbo Pascal применялись собственные драйверы для доступа к графическим режимам (в том числе полноцветным):

- **egavga.bgi** (640 × 200, 640 × 350, 640 × 480 16-цветные режимы),
- **vga256.bgi** (320 × 200 256-цветный режим),
- **ibm8514.bgi** (640 × 480 и 1024 × 768 256-цветные режимы).

Кроме того, была возможность использовать векторные шрифты.



Пример векторного шрифта в формате .chr от Borland

Источник: <http://kokovkin.narod.ru/rus/portfolio.htm>

В поставке Turbo Pascal были шрифты с засечками (Serif типа Times New Roman), без засечек (Sans Serif типа Arial и Calibri), моноширинные шрифты (типа Courier, используемые для отображения кода или имитации вывода печатной машинки — кстати, точечные шрифты для текстового режима по определению моноширинные), экзотические шрифты вроде рукописного (scri.chr) или готического (goth) написания.

Векторные шрифты из Turbo Pascal

Номер	Название	Файл	Краткое описание
1	TriplexFont	trip.chr	Трехобводный, прямой, с засечками
2	SmallFont	small.chr	Однообводный, прямой, моноширинный
3	SansSerifFont	sans.chr	Двухобводный, прямой, моноширинный
4	GothicFont	goth.chr	Готический
5	Script.chr	scri.chr	«Рукописный» шрифт, курсив
6	SimpleFont	simp.chr	Одноштриховый шрифт типа Courier Двухобводный, прямой, пропорциональный
7	TSGRFont	tscr.chr	Красивый наклонный шрифт типа Times Italic Трехобводный, курсив, с засечками
8	LCOMFont	lcom.chr	Шрифт типа Times New Roman
9	EuroFont	euro.chr	Шрифт типа Courier увеличенного размера
10	BoldFont	bold.chr	Крупный двухштриховый шрифт

Векторные шрифты хранят начертания шрифтов не в виде точек, а в виде набора векторов, задаваемых координатами. Каждый векторный шрифт поддерживает свое начертание, но для жирного,

курсивного или прямого начертания должен использоваться собственный набор. Векторные шрифты хорошо масштабируются.

Современные векторные шрифты (True Type Font .ttf, Post Script .ps и т.д.) строятся на похожих принципах.

Вернемся к точечным шрифтам. Возьмем, например, матрицу 8×8 :

	Начертание символа									Единицы и нули									Двоичная запись	Шестнадцатеричная запись
	1	2	3	4	5	6	7	8		1	2	3	4	5	6	7	8			
1										0	0	1	1	1	1	0	0		0011 1100	3C
2										0	1	0	0	0	0	1	0		0100 0010	42
3										1	0	0	0	0	0	0	1		1000 0001	81
4										1	0	1	0	0	1	0	1		1010 0101	A5
5										1	0	0	0	0	0	0	1		1000 0001	81
6										1	0	0	1	1	0	0	1		1001 1001	99
7										0	1	0	0	0	0	1	0		0100 0010	42
8										0	0	1	1	1	1	0	0		0011 1100	3C

Таким образом, символ в файле **.fnt** 8×8 в шестнадцатеричном виде будет представлен последовательностью:

3C 42 81 A5 81 99 42 3C

Если следующий символ представлял, допустим, кружок, его последовательность была следующей:

3C 42 81 81 81 81 42 3C

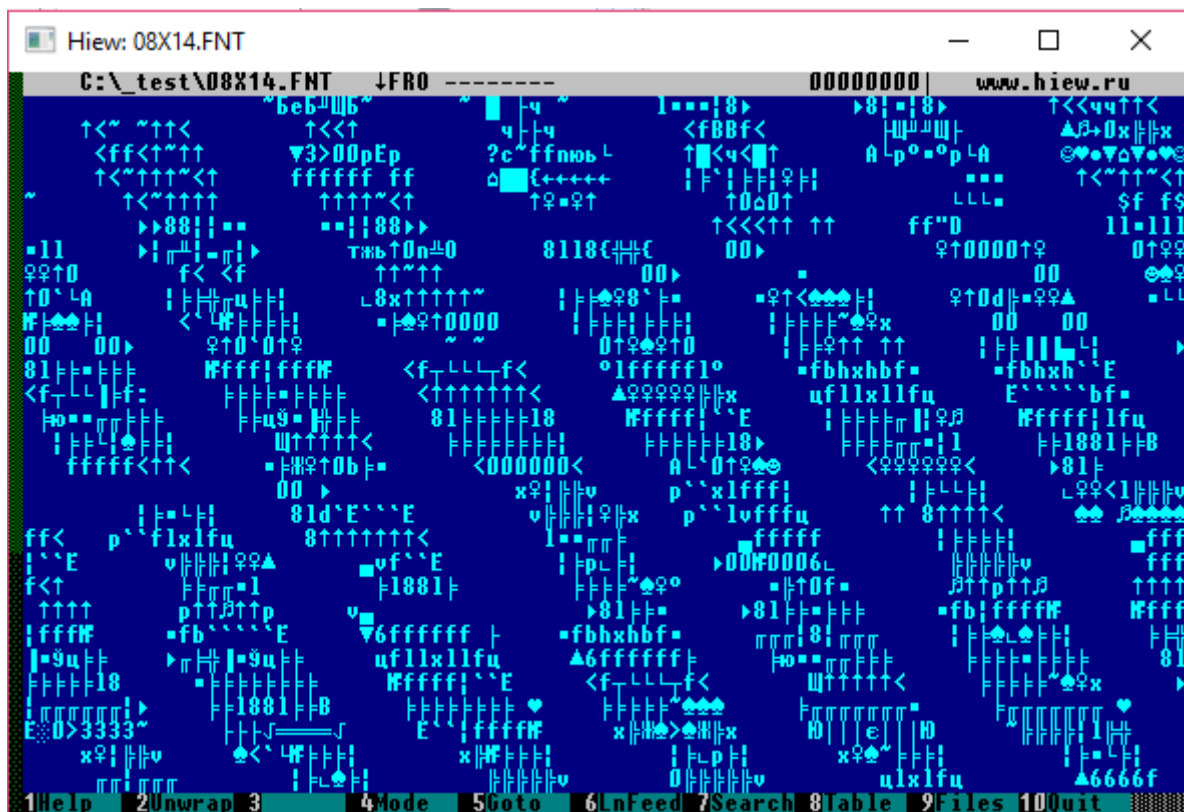
В файле все символы идут подряд:

3C 42 81 A5 81 99 42 3C 3C 42 81 81 81 81 42 3C

Сам файл **.fnt** — двоичный, то есть в него записываются числа в двоичном виде.

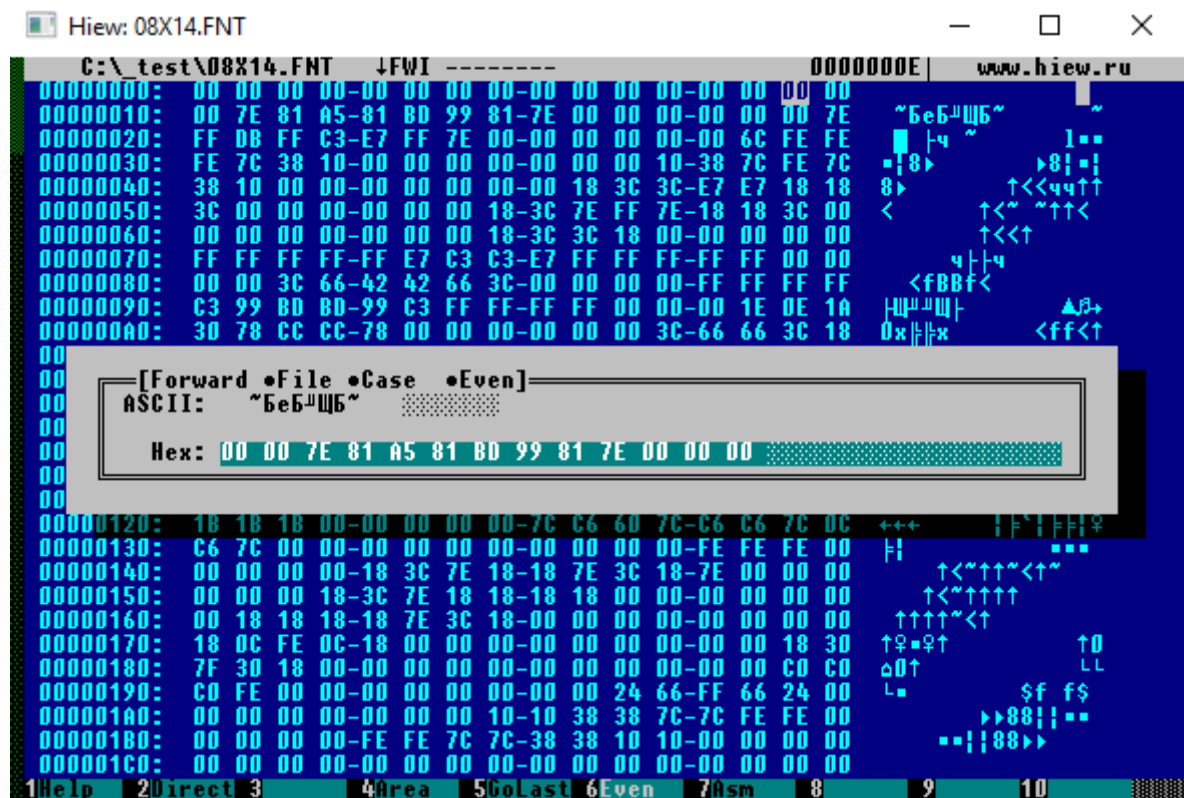
В MS DOS использовалась кодировка из 256 символов. Фактически заполнение файла **.fnt** и определяло ту кодировку, которая использовалась. Но если такая ситуация была характерна для точечных и векторных шрифтов, она не обязательно могла соблюдаться во всех случаях. Вспомните: первые машины использовали для ввода-вывода информации на печатную машинку. Некоторые устройства (калькуляторы, индикаторы на микроволновых печах, стиральных машинах и т. д.) используют семисегментные индикаторы. Каким образом используется кодирование в таких случаях?

Так выглядит шрифт **.fnt** при открытии на чтение:



Видны повторяющиеся последовательности, разделенные нулевыми символами. Это и есть точечные образы шрифтов.

Повторяющиеся паттерны формируют вполне узнаваемый рисунок. Колонки смещаются вправо, так как число отображаемых символов не кратно 14 (каждый «символ» на самом деле двухбайтное число, которое кодирует строку точек символа. Кстати, порядок байтов здесь не действует строго справа налево, так как это будет выводиться на экране).



Позиция первого символа имеет код 1 — в ASCII это управляющий символ, но если выводить символ на экран, отобразится значок смайла (сравните с представленным выше кодированием смайла).

Кодировки

Кодировки произошли от азбуки Морзе и кодов Бодо, в честь которого названа единица измерения передачи информации — бод. Сейчас все большее распространение получает Юникод:

- UTF-8 с переменной длиной, применяемый для передачи в интернете;
- UTF-16 с двумя байтами на символ, применяемый в Windows — ему присущи сложности, связанные с проблемой порядка байт и использованием специального символа Byte Order Mark — BOM);
- до этого применялись семибитные и восьмибитные кодировки.

Семибитная кодировка позволяла закодировать $2^7 = 127$ позиций, включая управляющие символы (перевод каретки, новая строка, звонок печатающей машинки, табуляция, возврат строки и т. д.) и алфавитно-цифровые символы одного языка (латинского).

ASCII Code Chart																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL

В отечественных машинах использовался транслитерированный вариант семибитной кодировки ASCII — KOI-7, в котором символы латиницы были транслитерированы в кириллицу, что позволяло работать и обеспечивать некоторую языковую совместимость (из транслита можно было понять, о чем идет речь, хотя это и было неудобно). В семибитной кодировке пришлось бы использовать русскоязычные команды и управляющие конструкции.

Семибитная кодировка является не совсем экономной, так как 1 бит в ней остается незадействованным. При этом добавление всего лишь одного бита позволяет увеличить таблицу в два раза.

Восьмибитная кодировка позволяет закодировать $2^8 = 256$ символов.

Если особенности ASCII более или менее понятны — по сути, первые 127 символов существуют в неизменном порядке до сих пор в большинстве кодировок, — то заполнение таблицы символов остается вопросом.

Например, можно заполнить таблицу недостающими символами европейских алфавитов (немецкого, исландского, французского, чешского) — всем хватит, кроме кириллических и иных шрифтов.

ANSI Extended ASCII (Windows)

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
8	□	□	,	f	//	...	†	‡	^	‰	Š	<	œ	□	□	□
9	□	`	'	“	”	•	–	—	~	™	š	>	œ	□	□	ÿ
A		ı	ı	£	¤	¥	¦	§	¨	©	ª	«	¬	–	®	—
B	°	±	²	³	´	µ	¶	·	¸	¹	º	»	¼	½	¾	¿
C	À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
D	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
E	à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
F	ø	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ

cpplus.com

Можно во вторую половину сложить то, что было в **koi7**, — получится **koi8r**:

±	℥	₹	₱	₪	₧	₦	₯	₰	₱	₶	₷	₸	₹	₺	₻	₼
128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	
₽	₾	₿	₿	₿	₿	₿	₿	₿	₿	₿	₿	₿	₿	₿	₿	₿
144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159	
₿	₿	₿	₿	₿	₿	₿	₿	₿	₿	₿	₿	₿	₿	₿	₿	₿
160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	
А	Б	В	Г	Д	Е	Ж	З	И	Й	К	Л	М	Н	О	П	
176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191	
Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я	
192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	
а	б	в	г	д	е	ж	з	и	й	к	л	м	н	о	п	
208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223	
р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	
224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	
Ё	ё	’	’	’	’	’	’	’	’	’	’	’	’	’	’	’
240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255	

Возможно, это не самый удобный вариант для работы: сортировка по алфавиту становится нетривиальной, а задача автоматической транслитерации — не самая необходимая.

Кодировка IBM (включая первые 127 символов):

[	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0		☺	☹	♥	♦	♣	♠	●		○						
1	▶	◀		!			—		↑	↓	→	←		↔	▲	▼
2		!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	'	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	□
8	Ç	ü	é	â	ã	ä	å	ç	ê	ë	è	í	î	ï	Ä	Å
9	Ê	æ	œ	ô	õ	ö	ù	û	ÿ	Ö	Ü	€	£	¥	ℳ	ƒ
A		í	ó	ú	ñ	ñ	ª	º	¿	¬	½	¼	¾	¼	»	»
B																
C																
D																
E	α	β	Γ	π	Σ	σ	μ	τ	φ	θ	Ω	δ	∞	φ	ε	η
F	≡	±	≥	≤			÷	≈	°	•	•	√	π	²	■	□

Кодировка символов, предложенная IBM (соответствует ASCII - кодировке)

В DOS применялась система кодовых страниц. Одна часть символов замещалась национальными символами, другая включала псевдографику, которая не менялась.

Пример таблицы 866, используемой в MS DOS (ее поддерживал драйвер **keyrus**):

А	Б	В	Г	Д	Е	Ж	З	И	Й	К	Л	М	Н	О	П
128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143
Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я
144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159
а	б	в	г	д	е	ж	з	и	й	к	л	м	н	о	п
160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175
☐	☐	☐													
176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191
┌	└	┐	┌	└	┐	┌	└	┐	┌	└	┐	┌	└	┐	┌
192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207
└	┐	┌	└	┐	┌	└	┐	┌	└	┐	┌	└	┐	┌	└
208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223
р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я
224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239
Ё	ё	Є	є	İ	ı	Ÿ	ÿ	°	•	•	√	№	¤	■	nbsp
240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255

В дальнейшем в Windows для графического отображения использовалась кодировка CP-1251, затем UTF-16. Если в CP-1251 не было символов псевдографики, то в UTF-16 есть не только псевдографические символы, но и графические изображения управляющих символов. К слову, они получили новые коды, а первые 32 символа по-прежнему используются как управляющие.

Управляющие и псевдографические символы

В досовских кодировках управляющие и псевдографические символы часто имели собственное начертание, но обработка символа осуществлялась в зависимости от используемого прерывания и функции:

- при выводе строки печать символа 7 выводила гудок на динамик (на экран ничего не выводилось),
- печать символа 8 приводила к перемещению курсора назад,
- 9 (табуляция) — к перемещению курсора на 8 позиций вправо,
- символ 10 служил для обозначения новой строки,
- 13 — для перевода строки.

Однако, например, утилита **ndd** (norton disk doctor) и подобные (например, **defrag**) использовали 7, 8, 9, 10 вместе с символами 176–178 для отображения на экране секторов диска, а в других программах — для рисования полос прокрутки. Символ 13 печатал нотку. Символ 27→ использовался для изображения полосы прокрутки, его появление в файле могло означать конец. Утилита **copy** без ключа **/b** обрезала бы данный файл до появления этого символа. Символ 28← на самом деле кодировал Escape. Служебные символы активно использовались и такими программами, как Lexicon, чтобы задавать новые страницы и другие элементы разметки.

Формат файлов может различаться в операционных системах. Так, в Windows для новой строки использовалась последовательность 13-го и 10-го символа (\r\n), а в Linux — только 10-й (\n). То же самое касается и передачи строк в системные вызовы. Обычно это передача длины и самого массива байт, однако возможны варианты и строк переменной длины. Например, при программировании в Си \0 (нулевой символ) может заканчивать строку, а некоторые функции MS DOS как конец строки воспринимали символ \$. Фактически к управляющим символам относится и BOM, если в файле используется Юникод. К слову, ряд функциональных кнопок генерирует двухбайтный код (для F1-F12), а некоторые — однобайтный. Нажатие на кнопку Backspace отправляет программе символ с кодом 8, но вместо его печати программа перемещает курсор. Esc генерирует код 27, Enter — код 13.

ASCII таблица с кодировкой CP866

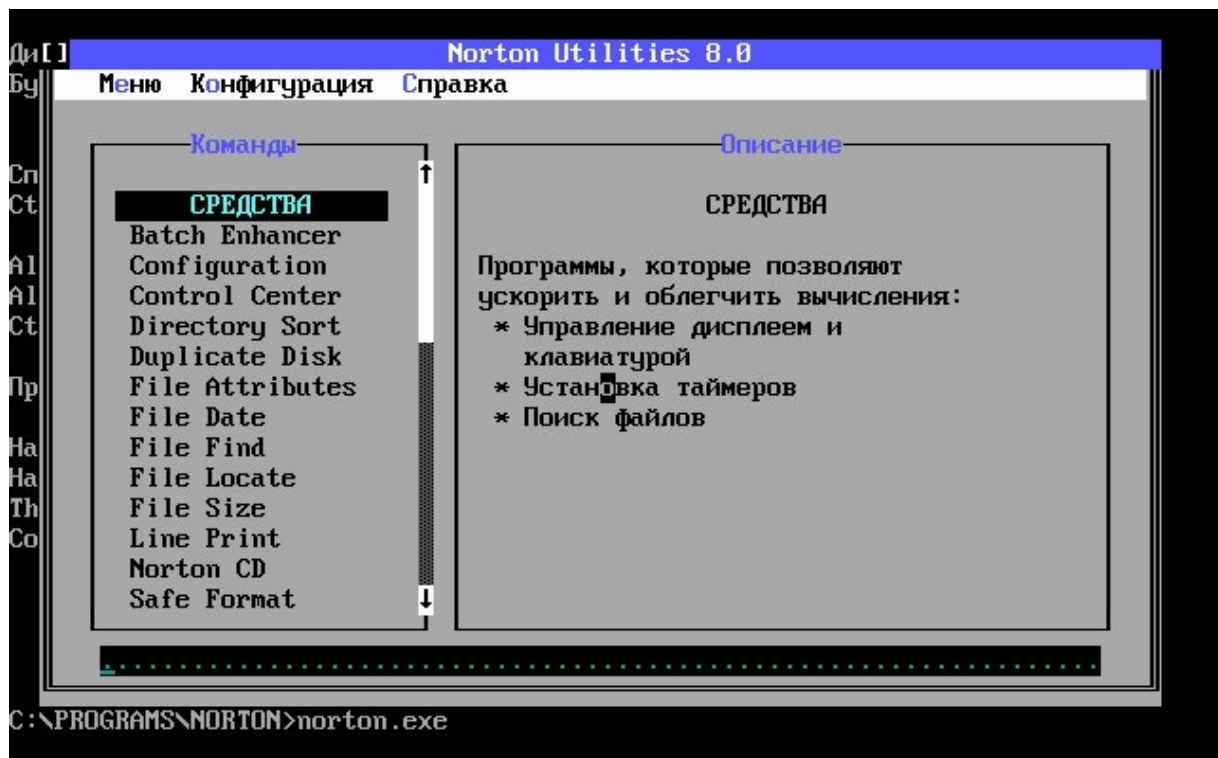
	00	10	20	30	40	50	60	70		80	90	A0	B0	C0	D0	E0	F0
0		▶		0	1	2	3	4	0	А	Б	В	Г	Д	Е	Ж	З
1	␣	␢	!	1	А	Q	а	q	1	Б	С	б	␣	␣	␣	с	±
2	␣	␣	"	2	В	R	Ь	г	2	В	Т	в	␣	␣	␣	т	>
3	♥	!!	#	3	С	S	о	s	3	Г	У	г		␣	␣	у	<
4	♦	␣	\$	4	Д	T	д	t	4	Д	Ф	д	␣	␣	␣	ф	г
5	␣	§	%	5	Е	U	е	u	5	Е	Х	е	␣	␣	␣	х	␣
6	␣	␣	&	6	Ф	V	ф	v	6	Ж	Ц	ж	␣	␣	␣	ц	␣
7	•	␣	'	7	Г	W	г	␣	7	З	Ч	з	␣	␣	␣	ч	␣
8	␣	†	<	8	Н	X	н	␣	8	И	Ш	и	␣	␣	␣	ш	␣
9	␣	↓	>	9	И	Y	и	␣	9	Й	Щ	й	␣	␣	␣	щ	␣
A	␣	→	*	:	␣	Z	␣	z	A	К	Ъ	к	␣	␣	␣	ъ	␣
B	␣	←	+	;	К	␣	␣	␣	B	Л	М	л	␣	␣	␣	м	␣
C	␣	␣	,	<	␣	␣	␣	␣	C	Н	Ь	н	␣	␣	␣	ь	␣
D	␣	␣	-	=	␣	␣	␣	␣	D	О	Э	о	␣	␣	␣	э	␣
E	␣	␣	.	>	␣	␣	␣	␣	E	О	Ю	о	␣	␣	␣	ю	␣
F	␣	␣	/	?	␣	␣	␣	␣	F	П	Я	п	␣	␣	␣	я	␣

Возможность переопределения таблицы символов (в том числе и в реальном времени) вкупе с начертанием «управляющих символов» и псевдографических давали возможность рисовать

псевдографические интерфейсы и даже псевдографический курсор мыши (вместо инвертированного цвета и фона знакоместа). Фактически мышь работала в пиксельном режиме, и несколько символов псевдографики перерисовывались в реальном времени: вычислялась позиция перекрываемых символов и на их месте отрисовывались новые из псевдографики с наложенным курсором мыши.



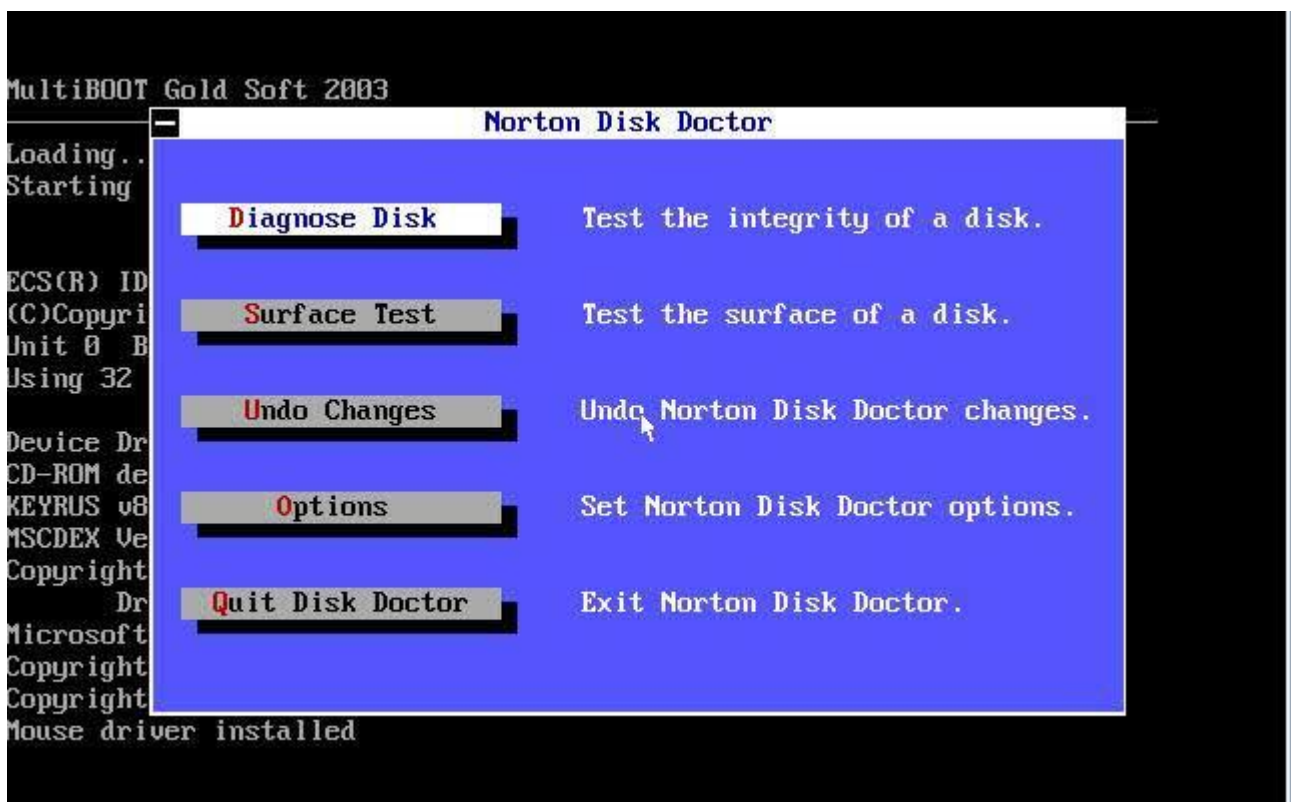
Псевдографические символы и начертания управляющих используются для формирования интерфейса в Norton Commander. Существует свободная библиотека **ncurses**, которая позволяет с использованием псевдографических символов формировать интерфейсы в консольных программах и встраиваемых системах (при наличии экрана).



Псевдографика в Norton Utilities:

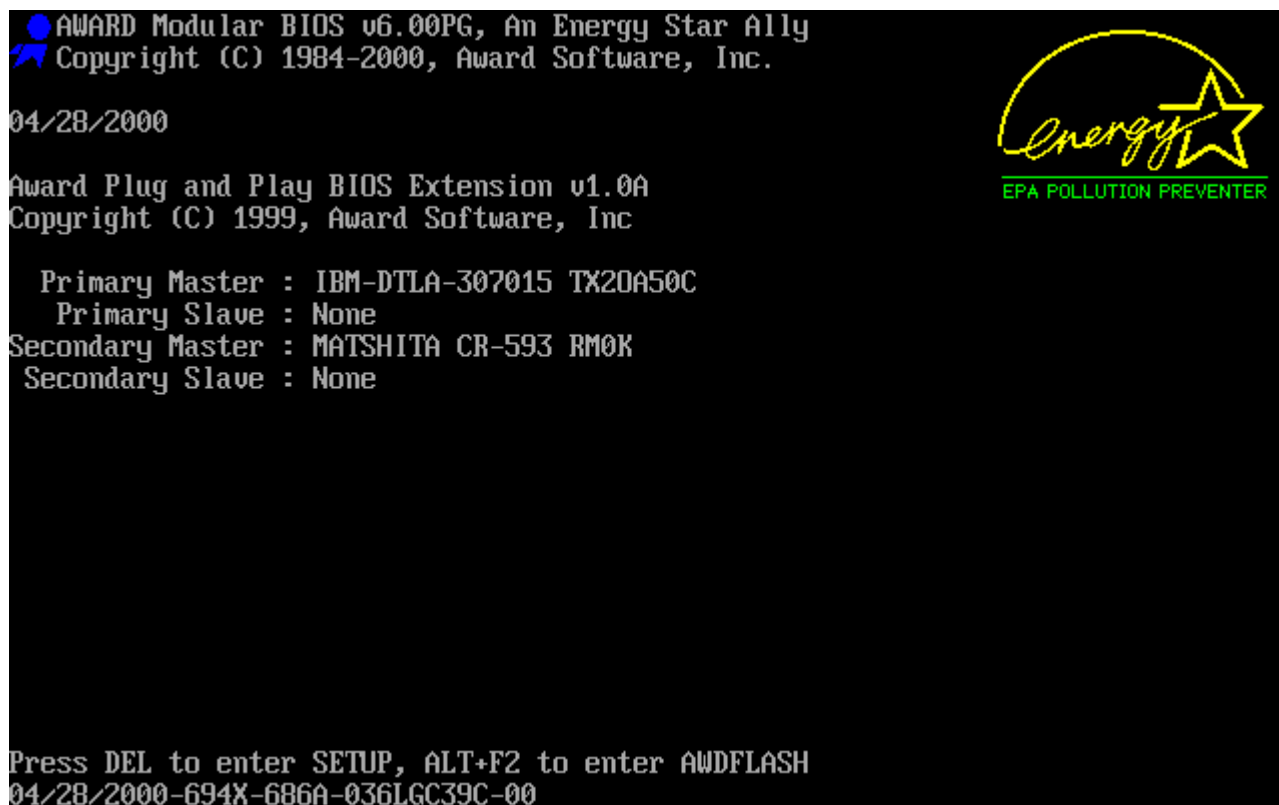


Псевдографика в Norton Utilities в текстовом режиме с переопределением знакогенератора (сравните с верхней картинкой):



Перед нами пример переопределения символов для более красивого интерфейса в NDD. Обратите внимание на псевдографический курсор мыши: он перемещался попиксельно. На месте **e** на экране напечатан не символ **e**, а псевдографический символ, полученный из матрицы символа **e**, с наложением фрагмента курсора в соответствии с его положением.

Переопределение символов позволяет включать в текстовом режиме картинки (не путать с ASCII-арт — там используется существующий набор символов).



Обратите внимание на картинку: они отрисованы в текстовом режиме благодаря переопределению таблиц символов.

Модульность DOS

Итак, DOS — не столько операционная система с ярко выраженными системными и пользовательскими интерфейсами, сколько конструктор. Программы в ней могли работать с оборудованием напрямую, могли использовать собственные драйверы и переопределять прерывания.

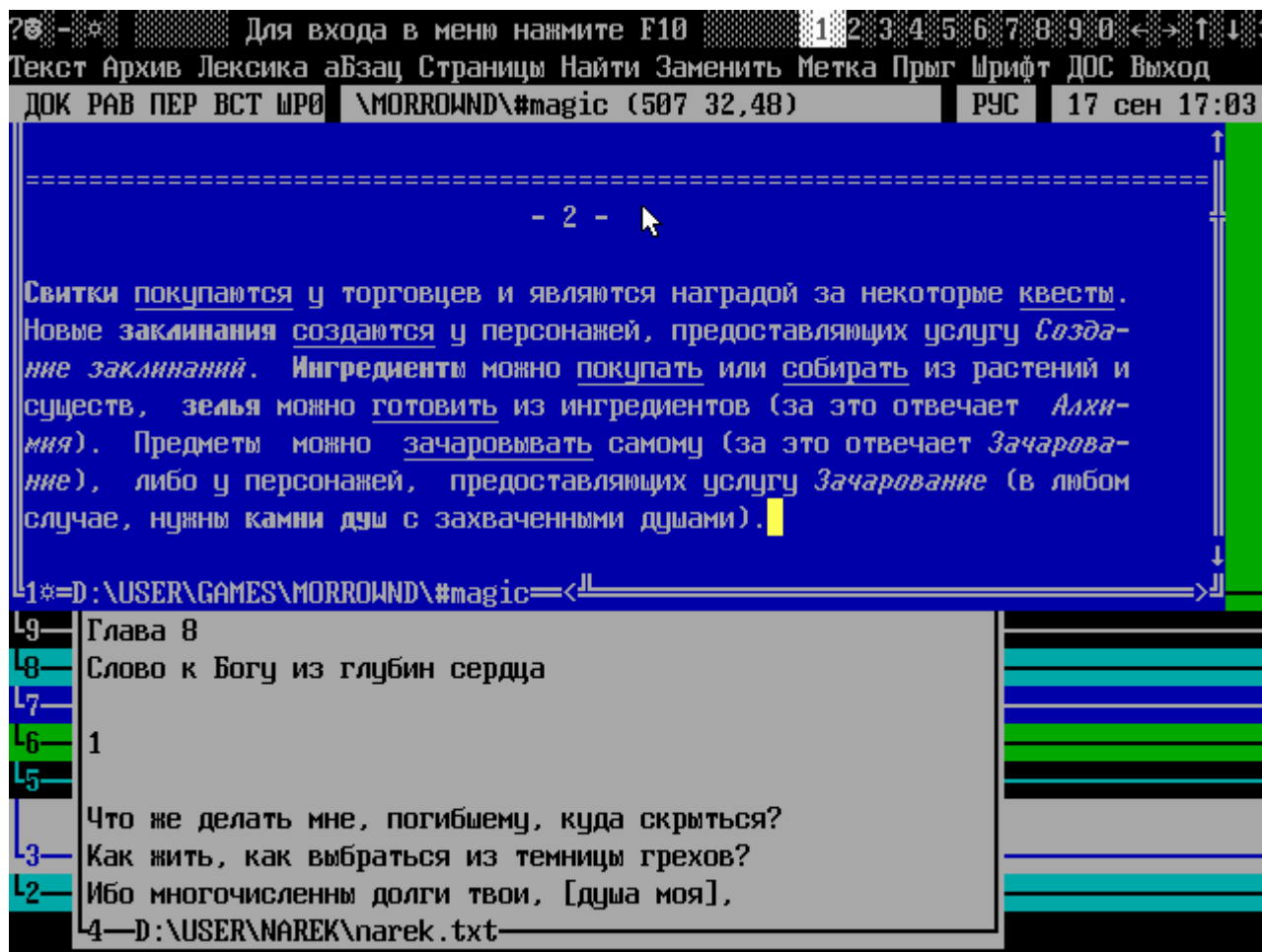
Текстовый процессор Lexicon использовал собственные драйверы для экрана и шрифтов, Norton Utilities — свои наборы шрифтов и драйвер мыши, игры могли полностью самостоятельно взаимодействовать с графикой на аппаратном уровне. Эти программы сами были мини-операционными системами, оставляя DOS роль загрузчика или операционной системы в самой базовой части.

Такие программы, как Dosshell, Desqview и Windows 1, в дальнейшем сами эволюционировали в операционные системы. Windows вплоть до 3.11 (а в некоторой степени и 9x), с одной стороны, представляли собой программы-оболочки для DOS. С другой стороны, они использовали значительный набор собственных драйверов, системных вызовов и т. д., что позволяет говорить об операционной системе в операционной системе. Впрочем, DOS и впоследствии играла роль загрузчика Windows и консоли восстановления. Более того, такие загрузчики, как loadlin, позволяют загрузить из DOS даже Linux.

В списке драйверов мы не упомянули **himem.sys** и **emm386.exe**. Формально это драйверы, которые загружаются, если упомянуты в **config.sys**, фактически же ситуация с ними обстоит сложнее.

Так, начиная с версий MS DOS 4.01 — 5.0 при использовании менеджера памяти **EMM386** (и его аналогов от других разработчиков) операционная система MS DOS работает как задача в виртуальном режиме. **EMM386** в этом случае является подобием операционной системы защищенного режима, передавая большинство системных прерываний ядру MS DOS в виртуальной задаче. Более того, виртуальный режим процессора 8086 позволил уже и в Windows выполнять приложения MS DOS. **Himem.sys**, предназначенный для поддержки HMA, был необходим и для запуска Windows 9x. Что интересно, впервые поддержка High Memory Area появилась в Windows 2.0, стартовавшей из DOS, а

затем уже механизм загрузки MS DOS в HMA был реализован в MS DOS 5.0. В приложениях использовались и такие 32-битные расширители, как **QEMM**, **DOS/4GW** и другие, позволяющие использовать до 64 Мб расширенной памяти.



Многооконный текстовый редактор Лексикон (использует собственные драйверы шрифтов).

Однозадачность

Что такое многозадачность? Возможны несколько ее реализаций. Простейший случай — невытесняющая многозадачность. В MS DOS многие сложные программы, такие как Lexicon, Quick Basic, Turbo Pascal, позволяли сделать «выход в DOS»: программа останавливалась, запускалась новая копия **command.com**, и можно было запустить другую программу (обычно на Norton Commander памяти не хватало, но можно было использовать аналогичный, но более экономичный Volkov Commander). А после завершения управление передавалось в вызвавшую программу — это и есть однозадачный режим. Если в системе реализована возможность остановки и передачи управления другой программе с возвратом в остановленную — это невытесняющая многозадачность. Ее не так сложно реализовать и в MS DOS, написав драйвер, управляющий переключением задачами (по прерыванию с клавиатуры вызвать менеджер задач и передать управление в нужную задачу). Однако во время выполнения одной задачи другие простаивают. Вытесняющая многозадачность позволяет выделять программе уже не все время процессора полностью, а лишь квант времени, останавливая его не по нажатию специальной кнопки, а по таймеру или получению прерывания от оборудования.

Интересна реализация многозадачности в MS DOS: фактически ее нет, но она может быть реализована благодаря механизму прерываний.

В 1988 году в Microsoft разрабатывалась MS DOS 4.0, которая в реальном режиме обладала вытесняющей многозадачностью, предназначенной для семейства процессоров 8086 (впоследствии

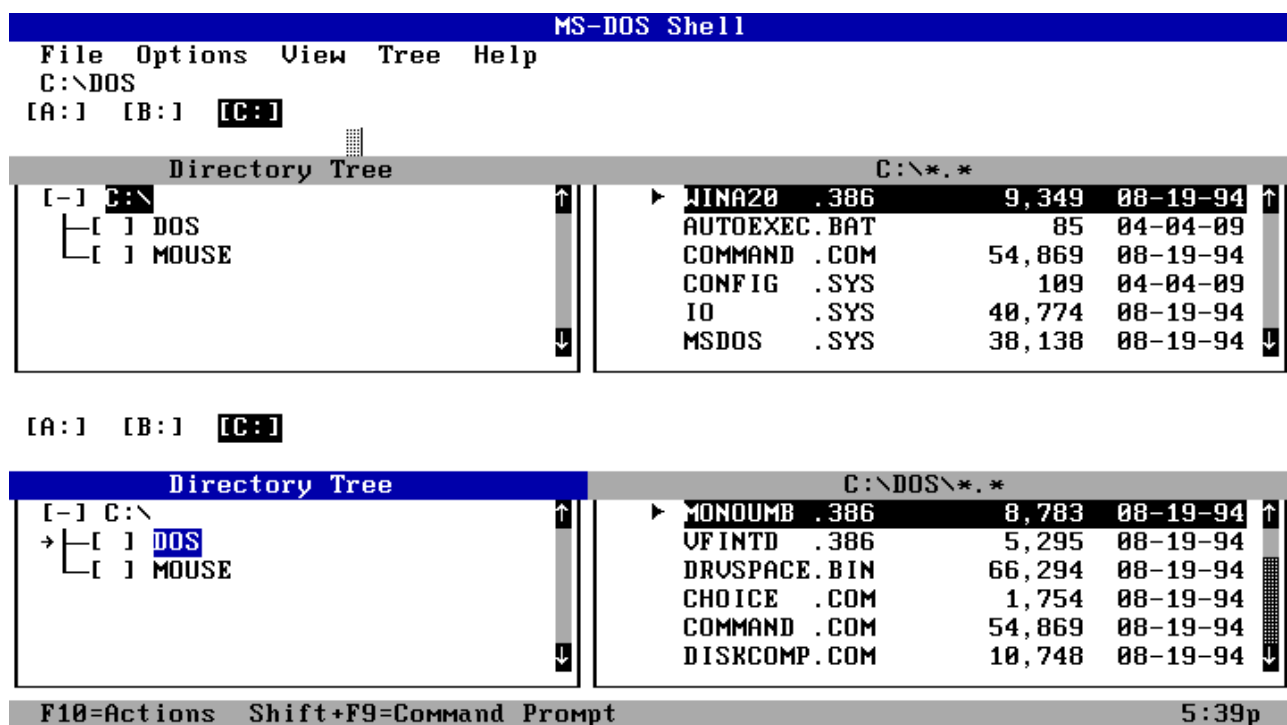
эта возможность была удалена). Еще она включала перемещаемые и выгружаемые сегменты памяти для кода и перемещаемые сегменты данных — менеджер памяти Windows был версией менеджера памяти DOS 4 — и имела возможность динамического переключения экранов. Версия была экспериментальной и в массовую продажу не пошла: под именем MS DOS 4.0 была выпущена ОС без этих возможностей. Впрочем, данные наработки, как уже было сказано, были использованы в Windows.

В версии 4.01, которая также вышла под именем MS DOS 4.0, имелась графическая среда DOS Shell с невывесняющей многозадачностью.

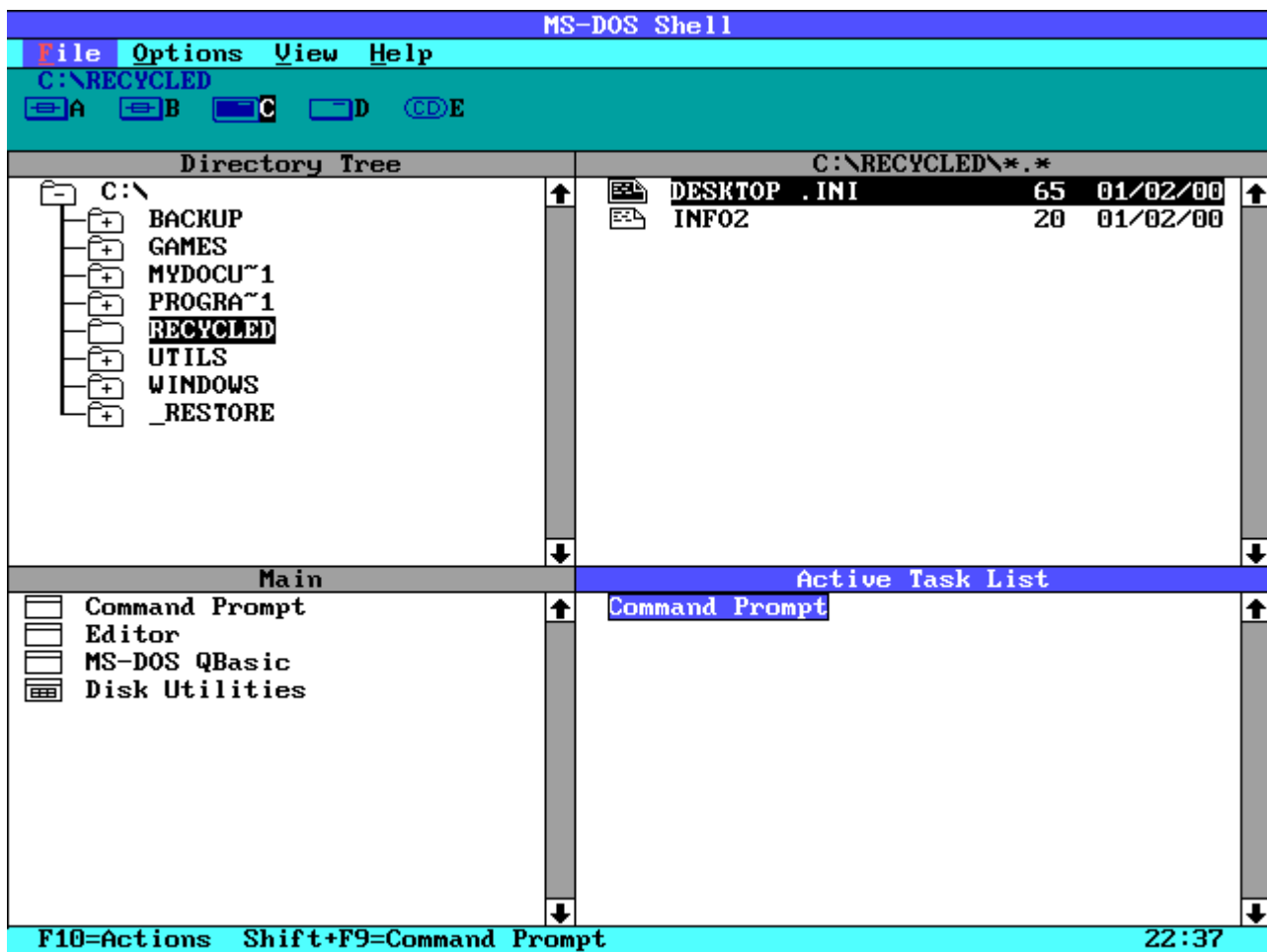
Существовали и альтернативные реализации многозадачности в MS DOS.

Интересно, что в MS DOS-совместимых операционных системах DR DOS 6.0 (1991 года) и Novell DOS 7.0 переключаемая многозадачность существовала. Ctrl-1, Ctrl-2 и т. д. позволяли переключаться между задачами (это напоминает переключение между терминалами в Linux: Alt-F1, Alt-F2), Ctrl-Esc — выйти на список задач. В Novell DOS 7.0 уже присутствовала вытесняющая многозадачность.

В 1980 сторонней компанией Quarterdeck Office Systems разрабатывалась оболочка для DOS DESQView, релиз которой последовал в 1985 году. Благодаря драйверу QEMM-386 (QEMM.COM) той же компании программа могла задействовать всю память. DESQView была сравнительно популярна, пока ее не вытеснили сначала DR DOS 6.0, а затем Microsoft Windows 3.11. В DESQView не было графического GUI, а попытка создать на ее основе DV/X с использованием X Window System успеха не принесла.



DOS Shell

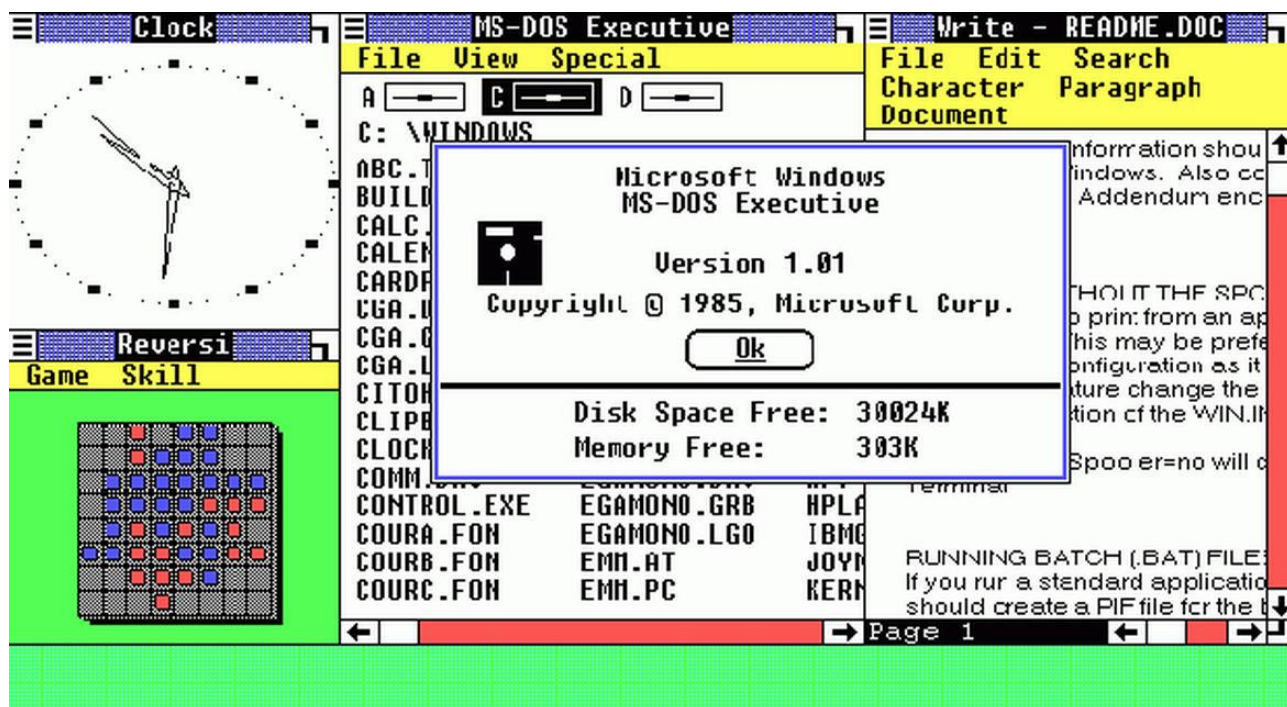


DOS Shell



DESQView — многооконность и кооперативная многозадачность

Microsoft Windows 1.0 появилась из необходимости иметь графический интерфейс и реализовать многозадачность. На это повлияли Xerox Star и Apple Lisa (1981–1982 годы), а также необходимость создать продукт для конкуренции с DESQView. Фактически это было приложение **win.com**, которое запускало оболочку MS DOS Executive. MS DOS Executive вытеснил и без того не пользовавшийся популярностью DOSshell.



Windows 1.01 (сравните с DOSShell)

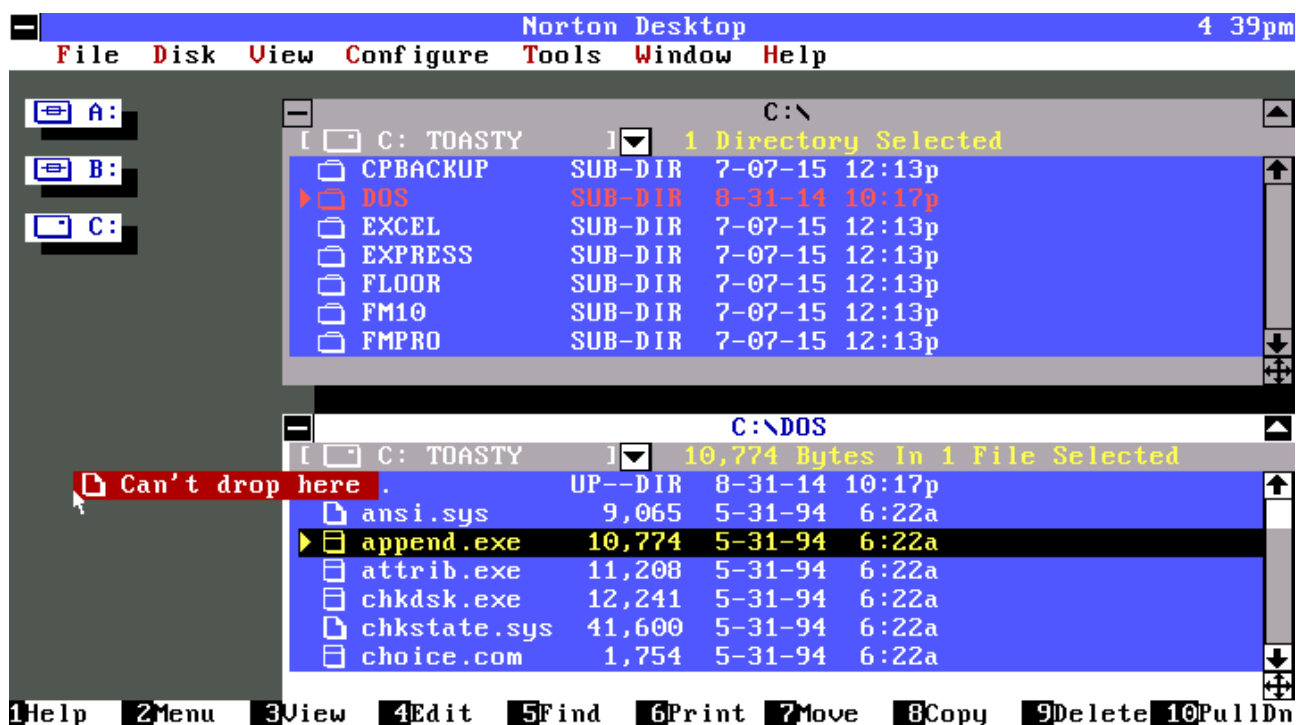
Из MS DOS Executive можно было выйти, вернув управление в однозадачный режим DOS. Подобная ситуация сохранялась довольно долго, в то время как Windows из приложения компонента MS DOS развивалась в отдельную систему, фактически поглощая оставшиеся компоненты DOS.

Оболочки

Оболочки для ОС могут представлять собой интерпретатор командной строки (Бейсик в многочисленных микрокомпьютерах на заре их появления, COMMAND.COM, bash), текстовый менеджер файлов с возможностью навигации по файлам, редактирования файлов и запуска программ. Популярен был такой интерфейс, как у Norton Commander: например, Norton Commander, Volkov Commander, DOS Navigator — для DOS, mc — Midnight Commander — для Linux, FAR для Windows. Оболочки могут иметь и альтернативную, зачастую псевдографическую концепцию.

Отдельно стоит рассматривать более сложные и многокомпонентные графические оболочки (Windows с Explorer, X11 Server с реализацией среды рабочего стола Gnome, KDE или Unity с оболочкой, например, Nautilus).

Оболочки сами по себе не обязаны реализовывать многозадачность, хотя DOSShell с этим справлялся, а Windows в принципе в чистом виде представляет собой не одну оболочку, а набор компонентов. В качестве примера оболочки без многозадачности наряду с Norton Commander можно привести оболочку от Symantec. Были попытки создать графическую версию для замещения встроенной оболочки в Windows 3.11. В принципе в MS DOS также вместо **command.com** можно запустить иную программу, от оболочки до альтернативных интерпретаторов командной строки, так как строка **shell=C:\ndos.com C:** в **config.sys** позволяла загрузить **ndos.com** — альтернативу командному интерпретатору от Norton Utilities. Так же и в Windows 3.11 вместо «Диспетчера программ» и в Windows 95 вместо Explorer, запускающихся по умолчанию, можно настроить запуск иной программы.



Norton Desktop

На наш взгляд, Windows 3.11 стоит считать отдельной от MS DOS системой с учетом того, что MS DOS фактически становился ее частью. Похожая ситуация сложилась с X Windows Server в Linux, хотя там немного иначе выглядит баланс между исходной операционной системой и графической системой (в Windows значительная часть функциональности тесно интегрирована с графической средой).

Утилиты

Утилиты — это полностью пользовательские программы, незаменимый компонент системы, не имеющий прямого отношения к ядру. В папке **msdos** операционной системы DOS содержалось большое число служебных программ. Сама MS DOS оказалась в какой-то степени гибридом CP/M и Unix, позаимствовав ряд решений, утилит, файловых конвейеров из UNIX. Ряд возможностей командного интерпретатора встроен в **command.com**, но большинство из них, как и в UNIX и Linux, — отдельные программы, имеющие расширение **.com** или **.exe**. Среди них были утилиты **fdisk**, **attrib**, **sys**, **label**, **chkdsk**, **undelete**, **subst**, **defrag**.

Иногда встроенных утилит было недостаточно и устанавливались дополнительные, такие как Norton Utilities, DOS Tools (утилиты для парковки головок винчестера и остановки системы — **park.com**, **shutdown.com**) и другие программы: упомянутые выше оболочки, редакторы (в том числе и такие, как **hiew**), языки программирования.

Примечательно, что в состав MS DOS входил редактор и интерпретатор диалекта языка Бейсик QBASIC. Он представлял собой урезанную версию коммерческого продукта Microsoft Quick Basic 4.5 без возможности компиляций и некоторых функций для работы с прерываниями — тем не менее подобные функции можно было реализовать самостоятельно благодаря возможности непосредственно выполнять Inline-код. Еще более интересно, что QBASIC.EXE был гораздо более важным компонентом MS DOS последних версий, чем это может показаться. Программы EDIT.COM и HELP.COM использовали встроенные редактор и справочную систему программы QBASIC.EXE, фактически просто запуская его с определенными ключами.

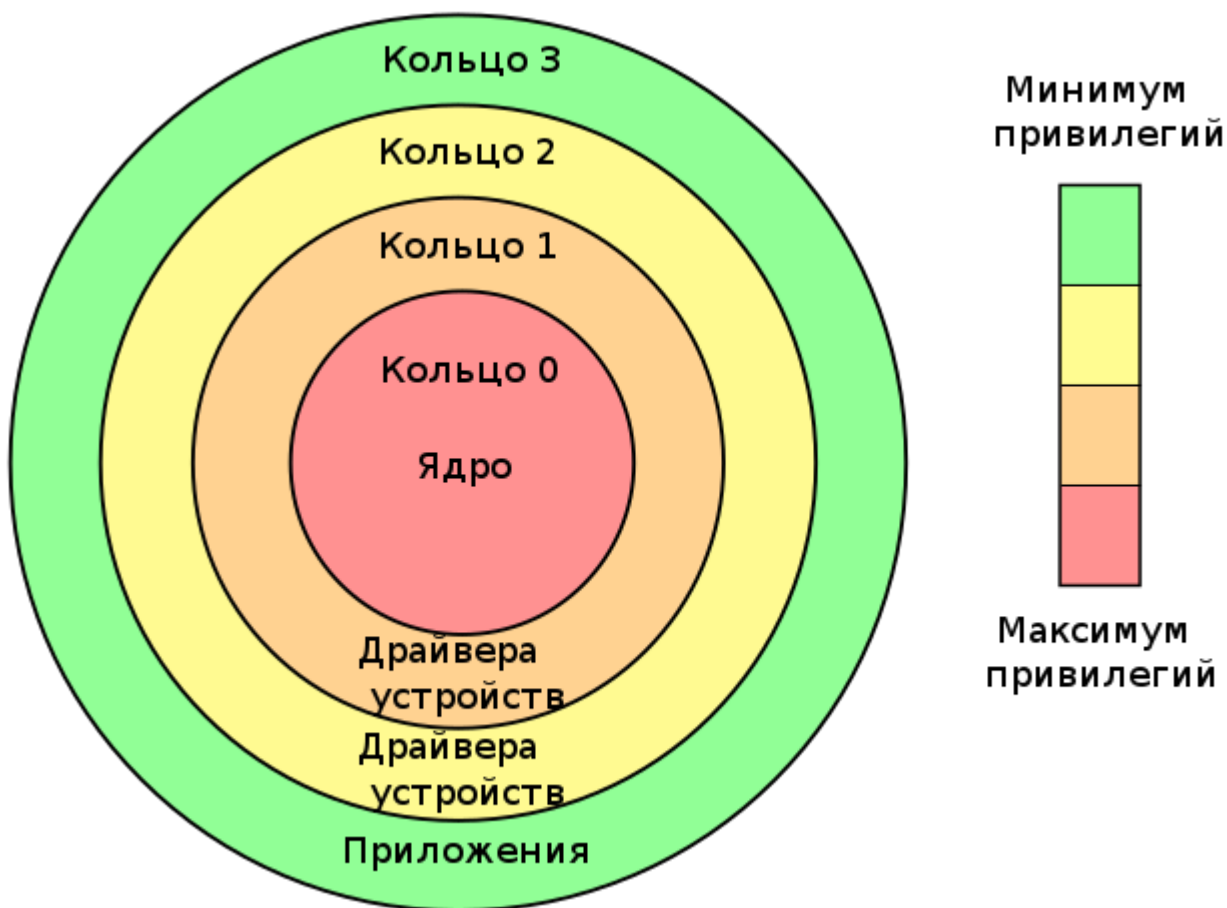
Архитектура ядра операционной системы

Процесс

Процесс — это именно то, что выполняет процессор. Но, как правило, под процессом понимается программа (программный код), работающая в выделенном процессором окружении, с соответствующими ресурсами (оперативной памятью, переменными окружения, открытыми файлами и т. д.) Когда мы запускаем программу на выполнение, обычно говорим, что в системе запущен соответствующий процесс. Чтобы обеспечить многозадачность, одна программа может сама запускать компоненты — дочерние процессы. Внутри процессов могут быть потоки — параллельно выполняемые участки кода, но работающие в общем адресном пространстве процесса и имеющие одинаковый доступ к его ресурсам. Разные процессы же в современных операционных системах изолированы (в MS DOS было только одно адресное пространство — и мы поймем почему), и могут общаться только через API операционной системы. Процессы более стабильны по сравнению с потоками, но для создания новых требуются большие затраты ресурсов операционной системы.

Кольца защиты процессора

В MS DOS не было никакого разграничения процессов: ошибка в приложении могла остановить всю операционную систему. Более того, реальный режим, в котором изначально работал MS DOS, не предоставлял механизмов защиты. Тем не менее, уже в Intel 80386 такие возможности возникли, появился защищенный режим (точнее, сначала он появился в Intel 80286, но в той форме, в которой мы его знаем сейчас был разработан именно в Intel 80386, с учетом проблем и недоработок в 80286 модели). Несмотря на то что многозадачность реализовывала операционная система (в принципе возможна и аппаратная реализация), для защиты процессов процессор предоставлял механизм колец защиты и страничную организацию памяти. Программы, предназначенные для незащищенного (так называемого реального) режима, в котором работала DOS, и защищенного режима, в общем случае несовместимы. В защищенном режиме невозможно использование прерываний BIOS и запуск программ, написанных для исполнения в реальном режиме. Тем не менее такую возможность дает виртуальный режим. 32-битный 80386 процессор умел аппаратно создавать «виртуальные машины», в которых мог выполняться 16-битный код реального режима. Это позволило очень долго обеспечивать совместимость с 16-битными DOS-приложениями. В 64-битных процессорах такая возможность была упразднена, так как старые приложения были уже не нужны, а также из-за того, что сам 64-битный процессор должен поддерживать и старый (теперь уже) 32-битный режим для совместимости.



В процессорах начиная с Intel 80386 и во всех современных существуют четыре уровня привилегий — четыре кольца защиты, пронумерованные от 0 (наиболее привилегированный уровень), до 3 (наименее привилегированный) по отношению к ресурсам, на которые распространяется действие механизмов защиты процессора. Среди них память, порты ввода-вывода и возможность выполнения некоторых инструкций. Каждую команду современный Intel-совместимый процессор выполняет, находясь на том или ином уровне привилегий: от этого зависит, что может и чего не может сделать код. Некоторые инструкции могут быть выполнены на уровне привилегий 0 и при попытке выполнения их на ином уровне приведут к вызову исключения. Также исключение будет вызвано при недопустимом с точки зрения защиты обращении к памяти или устройствам.

Несмотря на наличие четырех уровней, в современных операционных системах обычно используется всего два — 0 и 3, предназначенные для ядра и для пользовательского пространства. Фактически программа не может производить никакие действия с файлами, сетью или устройствами сама: для этого она обращается к функциям ядра. Возможность реализации тех или иных функций в уровне 0 или уровне 3 определяет архитектуру ядра. Возможность доступа приложений к уровню 0, пусть и в целях совместимости, делает систему нестабильной — для сравнения соотнесем ее с Windows 95.

Сравнивая Windows и Linux, часто говорят, что Windows зависает или крашится. На самом деле причины не в том, что архитектура Windows значительно хуже Linux. Более того, современные версии Linux принадлежат к архитектуре NT, которая является развитием операционной системы OS/2, разработанной совместно с IBM.

Windows «не повезло», что на момент появления новых решений, в частности механизмов защиты в 286, а затем кардинального пересмотра в процессоре 386, требовалось обеспечение совместимости для всего набора существующего ПО.

Когда Линус Торвальдс разработал ядро Linux, он писал его для процессора Intel 386 и не планировал поддержку для других процессоров, и уж тем более ранних моделей. Поэтому работа с разграничением возможностей для процессов в Linux шла «из коробки».

Поэтому Linux стабильная и надежная система. Кроме того, ядро открыто и распространяется по лицензии GNU. У ядра Linux более 1000 контрибуторов, и каждый из сообщества, кто находит в ядре ошибку, может ее исправить и отправить пулл реквест для внесения изменения. Собственно говоря, для совместной работы над ядром Линус Торвалдс и был придумал Git.

Что же касается приложений, они работают в кольце защиты с наименьшим приоритетом и не имеют доступа к чужой памяти и устройствам. С ресурсами они общаются через механизм системных вызовов и напрямую что-то испортить не могут. Максимум — программа может «зависнуть» сама, но тогда ее может остановить механизм операционной системы, не причиняя вреда другим программам.

Есть у работы с кольцами защиты и минусы. Процессор вынужден помнить, в какой момент и в каком режиме выполняются текущие команды. При переходе из одного режима на другой требуются ресурсы и время на переключение контекста. Поэтому некоторые критичные для обеспечения работы механизмы в некоторых операционных системах выполняются в нулевом конце защиты, в адресном пространстве ядра. Это касается, например драйверов устройств в Linux и реализации стека TCP/IP, чтобы не тратить драгоценное время на переключение между кольцами защиты. Но надо и понимать, что код, выполняемый в пространстве ядра может «испортить» всю операционную систему, потому он должен быть хорошо отлажен и проверен.

Надо отметить, что в составе ядра тоже могут выполняться компоненты кода ядра (а в монолитных ядрах и драйверы) параллельно. В Linux такие компоненты называются потоками ядра (реже — процессами ядра). От процессов пользовательского пространства они отличаются тем, что имеют общий доступ ко всем ресурсам в пространстве ядра и действительно больше похожи на потоки. Переключение между потоками ядра не такое дорогое, как между процессами или процессом и ядром в пользовательском пространстве, так как не требует переключения контекста и перехода на другое кольцо защиты процессора.

Системные вызовы

Как программам использовать функции, реализуемые ядром ОС? Для этого предназначены системные вызовы. Это специальный интерфейс, API, который позволяет использовать многие функции, уже написанные и включенные в состав ядра операционной системы. Для записи файла, печати на экран, получения данных из сети не обязательно и не нужно напрямую реализовать это взаимодействие с устройствами. Системные вызовы реализуются уже в BIOS. Но так как BIOS — медленная по сравнению с ОЗУ память (ПЗУ), то при старте ядра ОС оно замещает функции ввода-вывода на свои. Но при этом системные вызовы в BIOS также нужны, чтобы можно было стартовать ОС. Современные UEFI пошли дальше и могут в ПЗУ содержать уже целые легковесные операционные системы.

Несмотря на отсутствие реализации механизмов защиты ядра в DOS, здесь ядро присутствовало (и более того, выполняло роль ядра и для целого семейства Windows, от 1 до Millenium). Но иногда возможностей ядра не хватало, поэтому некоторые игры для быстрого действия работали с памятью напрямую (в пространстве пользователя 80386 не получилось бы, поэтому первое, что сделали в Microsoft, чтобы перейти от DOS к Windows, — реализовали поддержку графики и звука DirectX).

В ОС с использованием колец защиты пользовательское ПО в принципе не может работать с оборудованием — механизмы защиты процессора не позволяют. Но работать с устройствами надо. Поэтому приложения через механизмы системных вызовов обращаются к ядру ОС (либо его компонентам и драйверам в случае монолитного ядра), а уже компоненты ядра, работая в его пространстве, общаются с оборудованием. Это и надежно, так как пользовательское ПО не может нарушить работу системы, и удобно, так как это ПО становится проще для разработки и не требует реализации низкоуровневых механизмов работы с аппаратным обеспечением.

Технически системные вызовы реализуются через вызов соответствующего программного прерывания (у каждой ОС свои, в Linux это **int 80h**). Кроме того, в архитектуре intel 32 появился альтернативный механизм системного вызова, более быстрый **sysenter**. А в архитектуре i64 — **syscall**.

Архитектуры ядер

О вызове функций ядра мы уже имеем представление. Они реализуются через программный шлюз интерфейса системных вызовов — соответствующее программное прерывание. Например, в DOS `int 0x20` и выше, в Linux `Int 0x80`, а также инструкции, появившиеся в разное время: **`syscall/sysenter/vsyscall/vDSO`**, (более подробно — в [этой статье](#)).

Важно, что в этом случае процессор (как минимум при использовании прерывания либо механизмов **`syscall/sysenter`**) переходит на другой уровень защиты. В результате происходит переключение контекста, выполнение процессором ряда проверок и переход на другой режим. Эта операция трудозатратная, в целом замедляет быстроедействие, что может быть критично при выполнении ряда операций. Быстроедействие — это один из критериев, определяющих архитектуру процессора: монолитная архитектура наиболее быстрая, микроядерная, особенно в своей крайней вариации — в виде экзоядра — наименее быстрая. Второй критерий — это надежность и отказоустойчивость. Код, выполненный в пространстве ядра, может нарушить всю систему — как в DOS, поэтому монолитная архитектура теоретически наименее безопасная с точки зрения применения драйверов устройств. А код, выполненный в пользовательском пространстве, не может нарушить другие процессы и само ядро, но более медленно выполняется (зато микроядерная архитектура наиболее безопасна).

При выборе между быстроедействием и безопасностью приходится следовать той или иной архитектуре ядра.

Несмотря на формальную небезопасность монолитной архитектуры ядра Linux, сама операционная система надежна и устойчива. Недостаток успешно компенсируется открытой моделью распространения и доступа к коду — благодаря этому многие драйверы очень тщательно отлаживаются и проверяются, и риск краха ядра в Linux не выше, чем зависания операционной системы Windows.

Рассмотрим основные варианты архитектуры ядер операционных систем, в том числе уже упомянутые монолитную и микроядерную.

Монолитное ядро

Все драйверы выполняются в пространстве ядра. Пример монолитного ядра — ядро Linux. Изначально драйверы требовалось скомпилировать в ядро, после чего дальнейшее развитие привело к появлению модульных ядер.

Если ошибка на уровне приложения приводит к остановке самого приложения, ошибка в драйвере ведет к краху всей системы. В Linux такое событие называется `Kernel panic`.

```

ide1: BM-DMA at 0xc008-0xc00f, BIOS settings: hdc:pio, hdd:pio
ne2k-pci.c:v1.03 9/22/2003 D. Becker/P. Gortmaker
http://www.scyld.com/network/ne2k-pci.html
hda: QEMU HARDDISK, ATA DISK drive
ide0 at 0x1f0-0x1f7,0x3f6 on irq 14
hdc: QEMU CD-ROM, ATAPI CD/DVD-ROM drive
ide1 at 0x170-0x177,0x376 on irq 15
ACPI: PCI Interrupt Link [LNKC] enabled at IRQ 10
ACPI: PCI Interrupt 0000:00:03.0[A] -> Link [LNKC] -> GSI 10 (level, low) -> IRQ
10
eth0: RealTek RTL-8029 found at 0xc100, IRQ 10, 52:54:00:12:34:56.
hda: max request size: 512KiB
hda: 180224 sectors (92 MB) w/256KiB Cache, CHS=178/255/63, (U)DMA
hda: set_multimode: status=0x41 { DriveReady Error }
hda: set_multimode: error=0x04 { DriveStatusError }
ide: failed opcode was: 0xef
hda: cache flushes supported
hda: hda1
hdc: ATAPI 4X CD-ROM drive, 512kB Cache, (U)DMA
Uniform CD-ROM driver Revision: 3.20
Done.
Begin: Mounting root file system... ...
/init: /init: 151: Syntax error: 0xf000=panic
Kernel panic - not syncing: Attempted to kill init!

```

Linux также использует возможности многозадачности — компоненты ядра запускаются как потоки ядра. Иногда можно встретить термин «процессы ядра», но технически они выполняются в одном адресном пространстве и могут повлиять друг на друга.

Модульное ядро

Модульное ядро — разновидность монолитного ядра, позволяющая подгружать и выгружать драйверы для выполнения в пространство ядра без компиляции. Современные дистрибутивы Linux используют модульное ядро.

Команды **insmod** и **modprobe** позволяют загружать и выгружать модули ядра, **lsmod** — посмотреть список загруженных модулей. **Insmod** работает с файлом ядра, **lsmod** оперирует зависимостями.

Микроядро

Микроядро — концепция, призванная решить проблему падения системы при крахе драйвера. В концепции микроядра драйверы выполняются в отдельном пространстве. Падение драйвера не приводит к краху ядра. По существу, ядро в данном случае служит лишь связующим звеном между множеством компонентов. Достоинство микроядра — его высокая надежность, недостаток — дополнительные затраты на переключение контекста.

Примерами микроядер можно считать ядро Minix 3 от Эндрю Таненбаума, ядро Mach, на котором базируется ряд известных ОС, исследовательская Microsoft Singularity. Также одной из лучших реализаций концепции микроядерной ОС является POSIX-совместимая ОС реального времени QNX.

На практике обычно применяются вариации на основе микроядра.

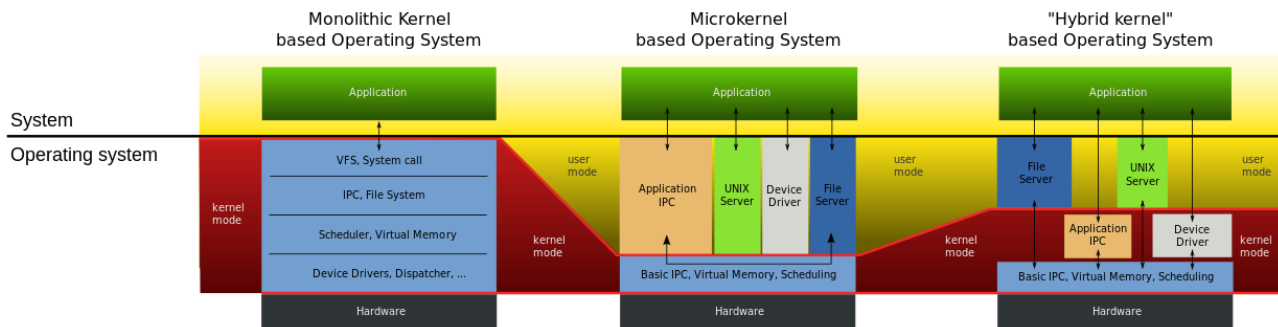
Экзоядро

Экзоядро — крайний случай микроядра, когда экзоядро занимается только взаимодействием процессов. Все функции работы с памятью, сетью, устройствами при этом вынесены в пользовательские библиотеки (libOS).

Наноядро

Наноядро — это разновидность ядра, реализующего только обработку прерываний. Его концепция близка к Hardware Abstraction Layer (HAL, Слой аппаратных абстракций). Такое ядро часто используется для виртуализации. Наноядро гостевой операционной системы может работать как процесс или модуль ядра другой операционной системы, позволяя запускать «неродные» приложения.

Гибридное ядро



Сравнение архитектуры

Гибридное ядро позволяет сгладить недостатки монолитного и микроядра. При этом часть критичных для производительности функций реализована внутри пространства ядра, а часть, как сервисы, в пользовательском пространстве.

Как примеры гибридных можно привести ядра, основанные на проекте Mach:

- проект MkLinux (предшественник Mac OS X) — ядро Linux запускалось поверх ядра Mach 3.0;
- проект (до сих пор незавершенный) GNU/Hurd для проекта GNU;
- XNU — гибридное ядро на базе микроядра Mach версии 2.5, компонентов от 4.3BSD и объектно-ориентированного интерфейса драйверов Driver Kit, разработанное для NextStep и в дальнейшем для Mac OS X.

Также отметим следующие примеры:

- **RTLinux** — микроядро, которое выполняет Linux как вытесняемый процесс (операционная система реального времени);
- **MinWin** — гибридное ядро, применяемое в современных Windows; состоит из NT-ядра и ряда компонентов пользовательского режима и драйверов.

Применение гибридного ядра приводит к тому, что, например, крах драйвера NVidia не вызывает перезагрузку драйвера операционной системой. С другой стороны, существует много других отказов, приводящих к появлению Blue Screen of Death (BSOD) — своего рода визитной карточки Windows.



Что-то пошло не так

Архитектура ядра Linux

Цифры и факты

- монолитное (модульное) ядро;
- около 30 тыс. файлов;
- около 8 млн. строк кода, не считая комментариев;
- репозиторий занимает около 1 Гб;
- linux-2.6.33.tar.bz2: 63 Mb;
- patch-2.6.33.bz2: 10Mb, около 1,7 млн измененных строк;
- около 6000 человек, чей код есть в ядре;
- ядро создал в 1991 году студент университета Хельсинки Линус Торвалдс;
- в качестве платформы он использовал ОС Minix, запущенную на персональном компьютере с процессором Intel 80386;
- в качестве примера для подражания Линус Торвалдс использовал ОС семейства Unix, а как путеводитель — сначала стандарт POSIX, а затем просто исходные коды программ из комплекта GNU (bash, gcc и пр);

- для работы над ядром Линус Торвальдс также создал Git.

Системные вызовы

Системные вызовы возможны как через вызов прерывания **int** (и выходом из него **iret**), так и с помощью новых инструкций **sysenter** (32 бита) и **syscall** (64 бита) и функции выхода из обработчика **sysexit**.

Системные вызовы предоставляют интерфейс, используемый прикладными программами — это API ядра. Большинство системных вызовов Linux взяты из стандарта POSIX, но есть и специфические для Linux системные вызовы.

Отметим различие между архитектурой системных вызовов Linux и UNIX с одной стороны и Windows — с другой. Дизайнеры Unix предпочитают предоставлять десять системных вызовов с одним параметром вместо одного с двадцатью. Классический пример — создание процесса. В Windows функция для создания процесса — **CreateProcess()** — принимает 10 аргументов, из которых 5 — структуры. В противоположность этому Unix-системы предоставляют два системных вызова (**fork()** и **exec()**), при этом первый — без параметров, второй — с тремя параметрами.

Управление памятью

В Linux используется плоская (**flat**) модель памяти, когда код и данные используют одно и то же адресное пространство.

Ядро Linux использует в качестве минимальной единицы памяти страницу. Размер страницы может зависеть от оборудования: на **x86** это 4Кб. Для хранения информации о странице физической памяти (ее физический адрес, принадлежность, режим использования и пр.) применяется специальная структура **page** размером в 40 байт.

Ядро использует возможности современных процессоров для организации виртуальной памяти. Благодаря манипуляциям с каталогами страниц виртуальной памяти каждый процесс получает адресное пространство размером 4 Гб (на 32-разрядных архитектурах). Часть этого пространства доступна процессу только на чтение или исполнение: туда отображаются интерфейсы ядра.

Важно, что процесс, работающий в пространстве пользователя, в большинстве случаев «не знает», где находятся его данные: в ОЗУ или в разделе подкачки (swap). Процесс может запросить у системы выделить ему память именно в ОЗУ, но система не обязана удовлетворять такую просьбу.

Управление процессами

Ядро Linux изначально имело поддержку вытесняющей многозадачности. В реализации вытесняющей многозадачности ядро выделяет каждому процессу определенный квант процессорного времени и «насильно» передает управление другому процессу по истечении этого кванта. С одной стороны, это создает накладные расходы на переключение режимов процессора и расчет приоритетов, с другой — повышает надежность и производительность.

Переключение процессов в Linux может производиться по наступлении двух событий: аппаратного прерывания или прерывания от таймера. Частота прерываний таймера устанавливается при компиляции ядра в диапазоне от 100 Гц до 1000 Гц. Как правило, аппаратные прерывания вызываются при нажатии клавиши на клавиатуре, движении мыши, поступлении сигналов от других устройств. Начиная с версии 2.6.23 появилась возможность собрать ядро, не использующее переключение процессов по таймеру. Это позволяет снизить энергопотребление в режиме простоя компьютера.

Планировщик процессов использует довольно сложный алгоритм, основанный на расчете приоритетов процессов. Среди процессов выделяются те, что требуют много процессорного времени, и те, что тратят больше времени на ввод-вывод. На основе этого регулярно пересчитываются приоритеты

процессов. Кроме того, в системе используются задаваемые пользователем значения **nice** для отдельных процессов.

Помимо многозадачности в режиме пользователя ядро Linux использует многозадачность в режиме ядра: само ядро многопоточно. Эти задачи видны при выводе **ps ax**: имена процессов отображаются в квадратных скобках.

PID	TTY	STAT	TIME	COMMAND
1	?	Ss	0:24	/sbin/init
2	?	S	0:00	[kthreadd]
3	?	S	0:07	[ksoftirqd/0]
5	?	S<	0:00	[kworker/0:0H]
7	?	S	0:29	[rcu_sched]
8	?	S	0:00	[rcu_bh]
9	?	S	0:00	[migration/0]
10	?	S	0:11	[watchdog/0]
11	?	S<	0:00	[khelper]
12	?	S	0:00	[kdevtmpfs]
13	?	S<	0:00	[netns]
14	?	S<	0:00	[perf]
15	?	S	0:00	[khungtaskd]
16	?	S<	0:01	[writeback]
17	?	SN	0:00	[ksmd]
18	?	SN	0:06	[khugepaged]
19	?	S<	0:00	[crypto]
20	?	S<	0:00	[kintegrityd]
21	?	S<	0:00	[bioset]
22	?	S<	0:00	[kblockd]
23	?	S<	0:00	[ata_sff]
24	?	S<	0:00	[md]
25	?	S<	0:00	[devfreq_wq]
27	?	S	0:32	[kworker/0:1]
29	?	S	0:08	[kswapd0]
30	?	S	0:00	[fsnotify_mark]
31	?	S	0:00	[ecryptfs-kthrea]
43	?	S<	0:00	[kthrotld]
44	?	S<	0:00	[acpi_thermal_pm]
45	?	S	0:00	[scsi_eh_0]
46	?	S<	0:00	[scsi_tmf_0]
47	?	S	0:00	[scsi_eh_1]
48	?	S<	0:00	[scsi_tmf_1]
51	?	S<	0:00	[ipv6_addrconf]
73	?	S<	0:00	[deferwq]

*Вывод команды **ps ax**. В квадратных скобках — потоки ядра*

Традиционные ядра Unix-систем не были вытесняемыми. Так, если процесс **/usr/bin/cat** запрашивает системный вызов **open()** для открытия файла **/media/cdrom/file.txt**, управление передается ядру. Ядро обнаруживает, что файл расположен на CD-диске, и начинает инициализацию привода (раскручивание диска и пр). Это занимает время, в процессе которого управление не возвращается пользовательским процессам, т. к. планировщик неактивен в то время, когда выполняется код ядра. Все пользовательские процессы ждут завершения этого вызова **open()**.

Современное ядро Linux, напротив, полностью вытесняемо. Планировщик отключается лишь на короткие промежутки времени, когда ядро никак нельзя прервать — например, на время инициализации устройств, требующих действий с фиксированными задержками. В любое другое время поток ядра может быть вытеснен, и тогда управление передается другому потоку ядра или пользовательскому процессу.

Сетевая подсистема

Несмотря на то, что сетевая подсистема могла бы выполняться в пространстве пользователя, на практике стек TCP/IP реализуется на уровне ядра. Сделано это для увеличения производительности, так как множественные обращения к ядру из пространства пользователя, неизбежные при формировании и анализе пакетов, требовали значительных накладных расходов.

Сетевая подсистема Linux обеспечивает функциональность, в которую входят:

- абстракция сокетов;
- стеки сетевых протоколов (TCP/IP, UDP/IP, IPX/SPX, AppleTalk и многое другое);
- маршрутизация (routing);
- пакетный фильтр (модуль Netfilter);
- абстракция сетевых интерфейсов.

Несмотря на то, что архитектурно Linux ближе к System V, в которой использовался Transport Layer Interface (TLI), здесь используется механизм сокетов Беркли из BSD.

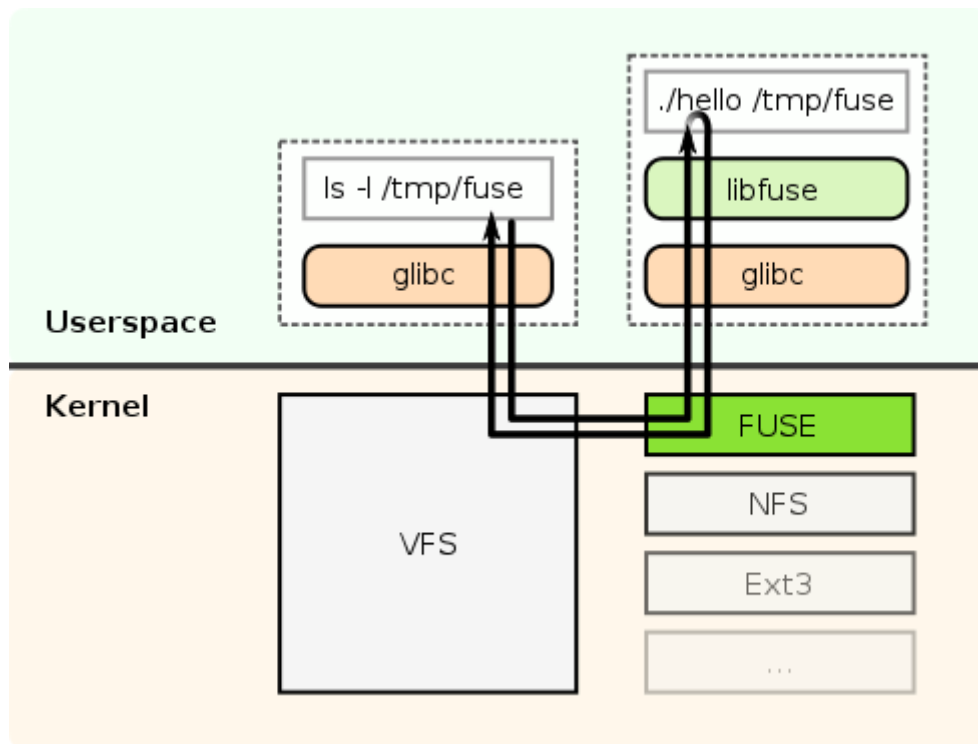
VFS — виртуальная файловая система

В отличие от Windows, в Unix-подобных ОС существует только одна файловая система, представляющая собой дерево директорий, растущее из «корня». Приложениям в большинстве случаев неважно, на каком носителе расположены данные файлов. Они могут находиться на жестком или оптическом диске, флеш-носителе или вообще на другом компьютере, доступном через механизмы сети. Подсистема, реализующая такой уровень абстрактности, называется виртуальной файловой системой (VFS).

В VFS файловые системы других носителей монтируются в одну из ее директорий. С другой стороны, благодаря файловой системе **proc**, смонтированной в **/proc**, любое приложение или скрипт может получить важные данные от ядра системы. Так, например, команда **cat /proc/version** позволяет получить информацию о версии ядра Linux в консоли или bash-скрипте.

Драйверы файловых систем

Благодаря архитектуре ядра Linux драйверы файловых систем относятся к более высокому уровню, чем драйверы устройств. Это связано с тем, что драйверы файловых систем не сообщаются ни с какими устройствами: драйвер файловой системы лишь реализует функции, предоставляемые им через интерфейс VFS. При этом данные пишутся и читаются в/из страницы памяти; какие из них и когда будут записаны на носитель — решает более низкий уровень. Благодаря этому был разработан драйвер FUSE (filesystem in userspace — «файловая система в пользовательском пространстве»), который делегирует функциональность драйвера файловой системы в модули, исполняемые в пространстве пользователя. Существует множество реализаций, позволяющих монтировать к VFS удаленные и распределенные файловые системы, архивы и т. д.



Механизм работы модуля FUSE

Автом: This file was made by User:SvenTranslation CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=3009564>

Страничный кеш

Страничный кеш — это подсистема ядра, которая оперирует страницами виртуальной памяти, организованными в виде базисного дерева (radix tree). Когда происходит чтение данных с носителя, данные читаются в выделяемую в кеше страницу, и она остается в там, а драйвер файловой системы читает из нее данные. Драйвер файловой системы пишет данные в страницы памяти, находящиеся в кеше. При этом страницы помечаются как «грязные» (dirty). Специальный поток ядра, **pdflush**, регулярно обходит кеш и формирует запросы на запись грязных страниц. Записанная на носитель грязная страница вновь помечается как чистая.

Уровень блочного ввода-вывода

Уровень блочного ввода-вывода — это подсистема ядра, которая оперирует очередями (**queues**), состоящими из структур **bio**. Каждая такая структура описывает одну операцию ввода-вывода (условно говоря, запрос вида «записать эти данные в блоки ##141–142 устройства **/dev/hda1**»). Для каждого процесса, осуществляющего ввод-вывод, формируется своя очередь. Из этого множества очередей создается одна очередь запросов к драйверу каждого устройства.

Планировщик ввода-вывода

Если выполнять запросы на дисковый ввод-вывод от приложений в том порядке, в котором они поступают, производительность системы в среднем будет очень низкой. Это связано с тем, что операция поиска нужного сектора на жестком диске — очень медленная. Поэтому планировщик обрабатывает очереди запросов, выполняя две операции:

- сортировку: планировщик старается ставить подряд запросы, обращающиеся к находящимся близко секторам диска;

- объединение: если в результате сортировки рядом оказались несколько запросов, обращающихся к последовательно расположенным секторам, их нужно объединить в один.

В современном ядре доступно несколько планировщиков: Anticipatory, Deadline, CFQ, noop. Существует версия ядра от Con Kolivas с еще одним планировщиком — BFQ. Планировщики могут выбираться при компиляции ядра либо при его запуске.

Отметим планировщик **noop**: он не выполняет ни сортировку, ни слияние запросов, а перенаправляет запросы драйверам устройств в порядке поступления. На системах с обычными жесткими дисками этот планировщик показал бы очень плохую производительность. Однако сейчас распространяются системы, в которых вместо жестких дисков используются флеш-носители. Для них время поиска сектора равно нулю, поэтому операции сортировки и слияния не нужны. В таких системах при использовании планировщика **noop** производительность не меняется, а потребление ресурсов несколько снижается.

Обработка прерываний

Практически все актуальные архитектуры оборудования используют для общения устройств с программным обеспечением концепцию прерываний. Выглядит это следующим образом: процессор исполняет процесс (неважно, поток ядра или пользовательский процесс), в ходе чего происходит прерывание от устройства. Процессор отвлекается от выполняемых задач и переключает управление на адрес памяти, сопоставленный данному номеру прерывания в специальной таблице прерываний. По этому адресу находится обработчик прерывания. Обработчик действует в зависимости от того, что именно произошло с этим устройством, затем управление передается планировщику процессов, который, решает, кому передать управление дальше.

Во время работы обработчика прерывания планировщик задач неактивен. Это неудивительно: предполагается, что обработчик прерывания работает непосредственно с устройством, а устройство может требовать выполнения каких-то действий в жестких временных рамках. Поэтому, если обработчик прерывания будет работать долго, все остальные процессы и потоки ядра будут ждать, а это обычно недопустимо.

В ядре Linux в результате любого аппаратного прерывания управление передается в функцию **do_IRQ()**. Эта функция использует отдельную таблицу зарегистрированных в ядре обработчиков прерываний, чтобы определить, куда передавать управление дальше.

Чтобы обеспечить минимальное время работы в контексте прерывания, в ядре Linux используется разделение обработчиков на верхние и нижние половины. Верхняя половина — это функция, которая регистрируется драйвером устройства в качестве обработчика определенного прерывания. Она выполняет только ту срочную работу. Затем она регистрирует другую функцию (свою нижнюю половину) и возвращает управление. Планировщик задач передаст управление зарегистрированной верхней половине, как только это будет возможно. При этом в большинстве случаев управление передается нижней половине сразу после завершения работы верхней. Нижняя работает как обычный поток ядра и может быть прервана в любой момент, и поэтому она имеет право исполняться как угодно долго.

В качестве примера рассмотрим обработчик прерывания от сетевой карты, сообщающий, что принят ethernet-пакет. Обработчик обязан выполнить два действия:

- взять пакет из буфера сетевой карты и сигнализировать ей, что пакет получен операционной системой, — это нужно сделать немедленно по получении прерывания, поскольку через миллисекунду в буфере будут уже совсем другие данные;
- поместить этот пакет в какие-либо структуры ядра, выяснить, к какому протоколу он относится, и передать его в соответствующие функции обработки — это нужно сделать как можно быстрее,

чтобы обеспечить максимальную производительность сетевой подсистемы, но не обязательно немедленно.

Первое действие выполняет верхняя половина обработчика, а второе — нижняя.

Драйверы устройств

Большинство драйверов устройств обычно компилируются в виде модулей ядра. Драйвер устройства получает запросы с двух сторон:

- от устройства — через зарегистрированные драйвером обработчики прерываний;
- от различных частей ядра — через API, который определяется конкретной подсистемой ядра и самим драйвером.

Практическое задание

Ответьте на следующие вопросы в Google Docs, максимально подробно объясняя, почему вы так считаете. Приложите ссылку на документ к уроку.

1. Что такое операционная система?
2. Что такое ядро операционной системы?
3. Что такое кольца защиты?
4. В чем плюсы монолитного ядра?
5. В чем минусы микроядра?
6. Что такое пространство ядра?
7. Почему MS DOS была нестабильной ОС?
8. Почему Linux считается ОС, которая не зависает?
9. Почему процессы в пространстве ядра работают быстрее?
10. Что такое системные вызовы?

Задачи со * предназначены для продвинутых учеников.

Дополнительные материалы

1. [Эволюция системных вызовов архитектуры x86.](#)
2. [Код загрузчика MSDOS 6.22.](#)
3. [Драйвер KeyRus на мемориальной странице Дмитрия Гуртяка.](#)
4. [Кольца, уровни защиты.](#)
5. [Работаем с модулями ядра в Linux.](#)
6. [Интервью с Эндрю Таненбаумом.](#)
7. [Еще интервью с Эндрю Таненбаумом.](#)
8. [Представление процессов в ядре Linux.](#)
9. [Peter Jay Salzman, Michael Burian, Ori Pomerantz The Linux Kernel Module Programming Guide.](#)
10. [Олег Цирюлик Модули Linux ядра.](#)

Используемая литература

Для подготовки данного методического пособия были использованы следующие ресурсы:

1. [Пример Hello world на ассемблере для DOS \(COM, EXE\) и Windows.](#)
2. [Устройство драйверов в MS DOS.](#)
3. [Процесс загрузки драйверов DOS.](#)
4. [Драйвер KeyRus на мемориальной странице Дмитрия Гуртяка.](#)
5. [Шрифты chr от Borland в Turbo Pascal.](#)
6. [Шрифты в графическом режиме Free Pascal.](#)
7. [Управление семисегментным индикатором.](#)
8. [Intel 80386.](#)
9. [MS DOS.](#)
10. [Защищенный режим.](#)
11. [Кольца защиты.](#)
12. [MinWin.](#)
13. [Ядро Linux за десять минут.](#)